ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ
ΠΡΟΓΡΑΜΜΑ ΜΕΤΑΠΤΥΧΙΑΚΩΝ ΣΠΟΥΔΩΝ
ΤΜΗΜΑΤΟΣ ΕΦΑΡΜΟΣΜΕΝΗΣ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΑΝΑΠΤΥΞΗ ΚΑΙ ΑΞΙΟΛΟΓΗΣΗ ΠΛΗΡΟΦΟΡΙΑΚΟΥ ΣΥΣΤΗΜΑΤΟΣ (ΔΙΑΧΕΙΡΙΣΗ ΚΤΗΝΙΑΤΡΕΙΟΥ) ΜΕ ΧΡΗΣΗ ΒΟΗΘΩΝ ΠΑΡΑΓΩΓΗΣ ΚΩΔΙΚΑ ΤΕΧΝΗΤΗΣ ΝΟΗΜΟΣΥΝΗΣ: ΜΙΑ ΜΕΛΕΤΗ ΠΕΡΙΠΤΩΣΗΣ ΕΣΤΙΑΣΜΕΝΗ ΣΤΗΝ ΑΞΙΟΛΟΓΗΣΗ ΤΗΣ ΠΟΙΟΤΗΤΑΣ ΚΩΔΙΚΑ ΚΑΙ ΤΗ ΣΤΡΑΤΗΓΙΚΗ ΔΙΑΔΙΚΑΣΙΑ PROMPTING

Διπλωματική Εργασία

του

Ευστάθιου Ιωσηφίδη

Θεσσαλονίκη, Ιανουάριος 2026

ΑΝΑΠΤΥΞΗ ΚΑΙ ΑΞΙΟΛΟΓΗΣΗ ΠΛΗΡΟΦΟΡΙΑΚΟΥ ΣΥΣΤΗΜΑΤΟΣ (ΔΙΑΧΕΙΡΙΣΗ ΚΤΗΝΙΑΤΡΕΙΟΥ) ΜΕ ΧΡΗΣΗ ΒΟΗΘΩΝ ΠΑΡΑΓΩΓΗΣ ΚΩΔΙΚΑ ΤΕΧΝΗΤΗΣ ΝΟΗΜΟΣΥΝΗΣ: ΜΙΑ ΜΕΛΕΤΗ ΠΕΡΙΠΤΩΣΗΣ ΕΣΤΙΑΣΜΕΝΗ ΣΤΗΝ ΑΞΙΟΛΟΓΗΣΗ ΤΗΣ ΠΟΙΟΤΗΤΑΣ ΚΩΔΙΚΑ ΚΑΙ ΤΗ ΣΤΡΑΤΗΓΙΚΗ ΔΙΑΔΙΚΑΣΙΑ PROMPTING

Ευστάθιος Ιωσηφίδης

Εφαρμοσμένη Πληροφορική - Πληροφοριακά Συστήματα, ΠΑΜΑΚ, 2024
Κτηνιατρική, ΑΠΘ, 2017

Διπλωματική Εργασία

υποβαλλόμενη για τη μερική εκπλήρωση των απαιτήσεων του

ΜΕΤΑΠΤΥΧΙΑΚΟΥ ΤΙΤΛΟΥ ΣΠΟΥΔΩΝ ΣΤΗΝ ΕΦΑΡΜΟΣΜΕΝΗ ΠΛΗΡΟΦΟΡΙΚΗ

Επιβλέπων/ουσα Καθηγητής/τρια
Αλέξανδρος Χατζηγεωργίου

Εγκρίθηκε από την τριμελή εξεταστική επιτροπή την __/__/____

Ονοματεπώνυμο 1	Ονοματεπώνυμο 2	Ονοματεπώνυμο 3
.....	.....	.....

Πληκτρολογήστε εδώ το ονοματεπώνυμο σας

Ευστάθιος Ιωσηφίδης
.....

Περίληψη

Η ραγδαία εξέλιξη των Μεγάλων Γλωσσικών Μοντέλων (Large Language Models - LLMs) έχει επιφέρει επαναστατικές αλλαγές στον κλάδο της Μηχανικής Λογισμικού, μετασχηματίζοντας τον παραδοσιακό ρόλο του προγραμματιστή. Παρόλο που η βιβλιογραφία βρίθει μελετών για την ικανότητα των Μεγάλων Γλωσσικών Μοντέλων (LLMs) να παράγουν μεμονωμένα τμήματα κώδικα (code snippets), παρατηρείται ερευνητικό κενό σχετικά με την εφικτότητα ανάπτυξης ολοκληρωμένων, σύνθετων πληροφοριακών συστημάτων σχεδόν αποκλειστικά με τη χρήση Τεχνητής Νοημοσύνης.

Η παρούσα διπλωματική εργασία εξετάζει αυτό ακριβώς το ζήτημα, έχοντας ως στόχο την ανάπτυξη και αξιολόγηση ενός Πληροφοριακού Συστήματος Διαχείρισης Κτηνιατρείου (Practice Information Management System - PIMS). Χρησιμοποιώντας το μοντέλο **Gemini 3 Pro** ως τον κύριο «συνεργάτη ανάπτυξης» και ακολουθώντας τη μεθοδολογία **Design Science Research (DSR)**, υλοποιήθηκε ένα πλήρες σύστημα βασισμένο σε τεχνολογίες Spring Boot (Backend), React (Frontend) και PostgreSQL.

Η αξιολόγηση του συστήματος πραγματοποιήθηκε μέσω στατικής ανάλυσης κώδικα με το εργαλείο **SonarQube** και μετρήσεων απόδοσης με το **Google Lighthouse**. Τα αποτελέσματα κατέδειξαν ότι το AI μπορεί να επιταχύνει τον χρόνο ανάπτυξης κατά περίπου 60-70%, παράγοντας λογισμικό υψηλής λειτουργικής αξιοπιστίας (Reliability Rating: A) και εξαιρετικής προσβασιμότητας (96/100). Ωστόσο, εντοπίστηκαν σημαντικές προκλήσεις: το Backend εμφάνισε κενά ασφαλείας (Security Rating: E) λόγω χρήσης σκληρά κωδικοποιημένων διαπιστευτηρίων, ενώ το Frontend παρουσίασε υψηλό ποσοστό επανάληψης κώδικα (~39%), υποδεικνύοντας αδυναμία του μοντέλου στην εφαρμογή αφαιρετικών αρχιτεκτονικών προτύπων.

Συμπερασματικά, η διατριβή αναδεικνύει ότι η ανάπτυξη End-to-End συστημάτων με ΤΝ είναι εφικτή και αποδοτική, υπό την προϋπόθεση ότι ο μηχανικός λογισμικού θα αναλάβει τον ρόλο του «Ορχηστρωτή», διαχειριζόμενος την αρχιτεκτονική συνοχή, την ασφάλεια και τις στρατηγικές Prompt Engineering.

Λέξεις-Κλειδιά: AI-Assisted Development, Large Language Models, Gemini 3 Pro, Code Quality, Design Science Research, PIMS.

Abstract

The rapid evolution of Large Language Models (LLMs) has brought revolutionary changes to Software Engineering, transforming the traditional role of the developer. While literature abounds with studies on the ability of Large Language Models (LLMs) to generate isolated code snippets, there is a distinct research gap regarding the feasibility of developing complete, complex Information Systems (End-to-End) almost exclusively using Artificial Intelligence.

This master's thesis addresses this issue by aiming to develop and evaluate a Practice Information Management System (PIMS). Using the **Gemini 3 Pro** model as the primary "development partner" and following the **Design Science Research (DSR)** methodology, a full-stack system was implemented based on Spring Boot (Backend), React (Frontend), and PostgreSQL technologies.

The system evaluation was conducted through static code analysis using **SonarQube** and performance metrics via **Google Lighthouse**. The results indicated that AI can accelerate development time by approximately 60-70%, producing software of high functional reliability (Reliability Rating: A) and exceptional accessibility (96/100). However, significant challenges were identified: the Backend exhibited security vulnerabilities (Security Rating: E) due to hardcoded credentials, while the Frontend showed a high rate of code duplication (~39%), suggesting the model's inability to apply abstract architectural patterns effectively.

In conclusion, the thesis highlights that End-to-End system development with AI is feasible and efficient, provided that the software engineer assumes the role of an "Orchestrator," managing architectural coherence, security, and Prompt Engineering strategies.

Keywords: AI-Assisted Development, Large Language Models, Gemini 3 Pro, Code Quality, Design Science Research, PIMS.

Περιεχόμενα

1. Εισαγωγή **1**
1.1 Πρόβλημα και σημαντικότητα του θέματος 1
1.2 Σκοπός και Στόχοι 1
1.3 Ερευνητικά ερωτήματα 2
1.4 Μεθοδολογική προσέγγιση 2
1.5 Διάρθρωση της μελέτης 2
2. Βιβλιογραφική επισκόπηση και θεωρητικό υπόβαθρο **3**
2.1 Τα Μεγάλα Γλωσσικά Μοντέλα (LLMs) στη Μηχανική Λογισμικού 3
2.1.1 Εξέλιξη, δυνατότητες και παραγωγικότητα 3
2.1.2 Κίνδυνοι ασφάλειας και ευπάθειες κώδικα 4
2.2 Μηχανική Υποβολής Ερωτημάτων (Prompt Engineering) για Προγραμματιστές 4
2.2.1 Η αναγκαιότητα της δομημένης εκπαίδευσης 4
2.2.2 Δομημένη ανάπτυξη κώδικα και Prompt Construction 5
2.2.3 Τεχνικές Prompting και πολυπλοκότητα 5
2.2.4 Μοτίβα Prompting (Prompt Patterns) 6
2.3 Ποιότητα λογισμικού και Μετρικές αξιολόγησης 6
2.3.1 Η διάσταση της λειτουργικότητας έναντι της δομής 7
2.3.2 Αυτοματοποιημένη παραγωγή μετρικών 7
2.3.3 Στατική ανάλυση και Τεχνικό χρέος 7
3. Ανάλυση και σχεδίαση συστήματος **8**
3.1 Ανάλυση επιχειρησιακών αναγκών 8
3.2 Λειτουργικές απαιτήσεις συστήματος 8
3.2.1 Υποσύστημα διαχείρισης πελατών και ασθενών 8
3.2.2 Υποσύστημα διαχείρισης ραντεβού 9
3.2.3 Ηλεκτρονικός Ιατρικός Φάκελος (EMR & SOAP) 9
3.3 Μη λειτουργικές απαιτήσεις 10
3.4 Αρχιτεκτονική και Τεχνολογίες 11
3.4.1 Backend: Spring Boot & Ροή Δεδομένων 11
3.4.2 Frontend: React.js 14
3.4.3 Βάση δεδομένων: PostgreSQL 15
3.4.4 Σχεδιασμός API και ροή δεδομένων (Data Flow) 15
3.5 Σχεδιασμός δεδομένων 16
3.5.1 Πίνακας Users (Προσωπικό Κλινικής) 17
3.5.2 Πίνακας Owners (Πελάτες / Κηδεμόνες) 18
3.5.3 Πίνακας Pets (Ασθενείς) 18
3.5.4 Πίνακας Visits (Ιατρικές Επισκέψεις - EMR) 18
3.6 Οριοθέτηση πεδίου πρωτοτύπου (Scope Definition) 18
4. Μεθοδολογία υλοποίησης και διαδικασία ανάπτυξης **20**
4.1 Περιβάλλον ανάπτυξης & Εργαλεία 20
4.2 Εκκίνηση έργου (Project Initialization) 21
4.3 Υλοποίηση Backend: Προκλήσεις και συνεργασία με την ΤΝ 21

4.3.1 Case Study 1: Διαχείριση συσχετίσεων Entity και JSON Serialization (The Infinite Recursion Problem) 21
4.3.2 Case Study 2: Η Στρατηγική Απόφαση αφαίρεσης του Lombok (Refactoring) 23
4.3.3 Case Study 3: Αντιμετώπιση παραισθήσεων της TN (AI Hallucinations) 25
4.4 Υλοποίηση ασφάλειας και διαχείριση πρόσβασης (Security & Authorization Challenges) 27
4.4.1 Η μάχη με τα CORS και CSRF: Η επικοινωνία Frontend-Backend 27
4.4.2 Το πρόβλημα των "403 Forbidden" και τα Request Matchers 28
4.4.3 Θωράκιση του JWT Filter: Από το Crash στη διαχείριση εξαιρέσεων 28
4.5 Ανάπτυξη διεπαφής χρήστη με λειτουργικά Prompts (Frontend Development) 30
4.6 Επίλυση προβλημάτων ενοποίησης (Integration Debugging) 32
4.7 Η TN ως Αρχιτέκτονας Λογισμικού. Η φάση του Antigravity 33
5. Αποτελέσματα και Αξιολόγηση 37
5.1 Αξιολόγηση επιδόσεων διεπαφής χρήστη (Frontend) 37
5.1.1 Συγκριτική ανάλυση μετρήσεων 38
5.1.2 Ανάλυση απόδοσης: Το ισοζύγιο λειτουργικότητας και ταχύτητας (Performance Trade-off) 39
5.1.3 Ποιότητα κώδικα και προσβασιμότητα (Quality & Accessibility) 40
5.1.4 Συμπέρασμα επιδόσεων 41
5.2 Αξιολόγηση ποιότητας κώδικα (Static Code Analysis) 41
5.2.1 Μεθοδολογία μέτρησης 41
5.2.2 Ανάλυση Backend (Spring Boot) 42
5.2.3 Ανάλυση Frontend (React) 44
5.2.4 Τεχνικό Χρέος (Technical Debt) και Συμπεράσματα 44
5.3 Μετρήσεις παραγωγικότητας και διαδικασίας ανάπτυξης 45
5.3.1 Ρυθμός επιτυχίας (Success Rate Analysis) 45
5.3.2 Ο Ρόλος της ανθρώπινης καθοδήγησης 47
5.3.3 Εκτίμηση χρόνου 47
6. Συμπεράσματα και μελλοντικές επεκτάσεις 48
6.1 Σύνοψη της έρευνας 48
6.2 Απαντήσεις στα ερευνητικά ερωτήματα 48
6.3 Συνεισφορά της διατριβής 49
6.4 Περιορισμοί της έρευνας (Limitations) 50
6.5 Μελλοντικές επεκτάσεις 50
ΒΙΒΛΙΟΓΡΑΦΙΑ (REFERENCES) 52
ΠΑΡΑΡΤΗΜΑ Α: ΚΡΙΣΙΜΑ ΤΜΗΜΑΤΑ ΚΩΔΙΚΑ 52
Α.1 Ρύθμιση Ασφάλειας (Security Configuration) 53
Α.2 Φίλτρο Αυθεντικοποίησης (JWT Filter with Error Handling) 54
Α.3 Axios Interceptor (Frontend-Backend Integration) 56
Α.4 Βασική Οντότητα χωρίς Lombok (Plain Java Entity) 57
ΠΑΡΑΡΤΗΜΑ Β: ΚΑΤΑΛΟΓΟΣ PROMPTS (PROMPT CATALOG) 60
ΠΑΡΑΡΤΗΜΑ Γ: ΚΑΤΑΛΟΓΟΣ USER STORIES 63
1. Ταυτοποίηση & Διαχείριση Χρηστών 63

1.1. Ταυτοποίηση (Authentication) 63
1.2. Διαχείριση Προφίλ 63
1.3. Διαχείριση Χρηστών (Μόνο Admin) 63
2. Διαχείριση Πελατών 63
2.1. CRUD Πελατών 63
2.2. Ειδικοί Ρόλοι 64
3. Διαχείριση Ασθενών 64
3.1. CRUD Ασθενών 64
3.2. Ταυτοποίηση & Κατάσταση 64
4. Διαχείριση Ραντεβού 64
4.1. Προγραμματισμός 64
4.2. Διαχείριση 64
5. Ιατρικά Αρχεία & Ιστορικό 65
5.1. Κλινικές Σημειώσεις 65
5.2. Ειδοποιήσεις 65

Κατάλογος Εικόνων

Εικόνα 3.1: Αρχικό διάγραμμα Περιπτώσεων Χρήσης (Use Case Diagram) 10

Εικόνα 3.2: Δομή Καταλόγων Backend Εφαρμογής (Project Structure) 13

Εικόνα 3.3: Δομή Καταλόγων Frontend Εφαρμογής (Project Structure) 15

Εικόνα 3.4: Διάγραμμα Οντοτήτων-Συσχετίσεων (ERD). 17

Εικόνα 4.1: Σφάλμα άπειρης αναδρομής JSON Serialization. 23

Εικόνα 4.2: Compilation Errors λόγω Lombok. 24

Εικόνα 4.3: Παράδειγμα Hallucination (λανθασμένο Property). 26

Εικόνα 4.4: Προβλήματα συμβατότητας Spring Security 6. 29

Εικόνα 4.5: Η αρχική μορφή της σελίδας εισόδου. 30

Εικόνα 4.6: Η τελική μορφή της σελίδας εισόδου. 31

Εικόνα 4.7: Η οθόνη διαχείρισης ιδιοκτητών. 31

Εικόνα 4.8: Σφάλματα CORS στον φυλλομετρητή. 32

Εικόνα 4.9: Αποτυχία εισόδου. 33

Εικόνα 4.10: Η φόρμα διαχείρισης ασθενών. 35

Εικόνα 4.11: Σφάλμα λογικής στο Frontend. 36

Εικόνα 4.12: Modal Ιστορικού Ασθενούς. 37

Εικόνα 5.1: Google Lighthouse Dashboard. 38

Εικόνα 5.2: Ανάλυση μεγέθους δέσμης κώδικα. 40

Εικόνα 5.3: Accessibility Score (96/100). 41

Εικόνα 5.4: SonarQube Backend Quality Gate. 43

Εικόνα 5.5: Technical Debt Comparison (Backend vs Frontend). 45

Κατάλογος Πινάκων

Πίνακας 4.1: Prompt για τη δημιουργία του Client Controller. 22

Πίνακας 4.2: Καταγραφή κρίσιμων σφαλμάτων Backend. 23

Πίνακας 4.3: Prompt για την αντικατάσταση του Lombok. 24

Πίνακας 4.4: Prompt διόρθωσης λανθασμένου Query (Hallucination) 26-27

Πίνακας 4.5: Prompt για την παραμετροποίηση του Spring Security 6. 29-30

Πίνακας 4.6: Απόσπασμα από το Σχέδιο Υλοποίησης του AI. 34-35

Πίνακας 4.7: Απόσπασμα από τον Οδηγό Επαλήθευσης (Walkthrough). 36-37

Πίνακας 5.1: Συγκριτική Αξιολόγηση Επιδόσεων (Lighthouse Metrics). 38-39

Πίνακας 5.2: Εξέλιξη Μετρήσεων Ποιότητας Backend. 42

Πίνακας 5.3: Εξέλιξη Μετρήσεων Ποιότητας Frontend. 44

Πίνακας 5.4: Συσχέτιση Τύπου Εργασίας και Απόδοσης AI. 46

Πίνακας Β.1: Κατάλογος Κρίσιμων Prompts (Prompt Catalog). 54-55

1. Εισαγωγή

1.1 Πρόβλημα και σημαντικότητα του θέματος

Η βιομηχανία λογισμικού βιώνει μια άνευ προηγουμένου αλλαγή παραδείγματος με την εισαγωγή των Βοηθών Κώδικα Τεχνητής Νοημοσύνης (AI Code Assistants). Εργαλεία όπως το GitHub Copilot, το ChatGPT και το Google Gemini ενσωματώνονται ταχύτατα στη ροή εργασίας των προγραμματιστών, υποσχόμενα αύξηση της παραγωγικότητας και μείωση του κόστους ανάπτυξης. Ωστόσο, η υπάρχουσα βιβλιογραφία εστιάζει κυρίως στην ικανότητα αυτών των μοντέλων να επιλύουν μεμονωμένα αλγοριθμικά προβλήματα ή να συμπληρώνουν μικρά τμήματα κώδικα. Λείπουν, σε μεγάλο βαθμό, εμπειρικές μελέτες που να εξετάζουν την ικανότητα των Μεγάλων Γλωσσικών Μοντέλων (LLMs) να κατασκευάζουν ολοκληρωμένα, πολυεπίπεδα πληροφοριακά συστήματα (End-to-End), διαχειριζόμενα την πολυπλοκότητα της αρχιτεκτονικής και της ενοποίησης υποσυστημάτων.

Παράλληλα, ο κλάδος της κτηνιατρικής ιατρικής αντιμετωπίζει τη δική του πρόκληση ψηφιακού μετασχηματισμού. Πολλές κτηνιατρικές μονάδες εξακολουθούν να βασίζονται σε απαρχαιωμένα συστήματα ή χειρόγραφες διαδικασίες, γεγονός που δυσχεραίνει τη διαχείριση του ιστορικού ασθενών και την εξυπηρέτηση των πελατών. Είναι επιτακτική η ανάγκη για σύγχρονα, προσβάσιμα και οικονομικά αποδοτικά συστήματα διαχείρισης κτηνιατρείων (PIMS). Η παρούσα διατριβή γεφυρώνει αυτά τα δύο πεδία, διερευνώντας αν η τεχνολογία ΤΝ μπορεί να αποτελέσει τον καταλύτη για την ταχεία και ποιοτική ανάπτυξη τέτοιων συστημάτων.

1.2 Σκοπός και Στόχοι

Κύριος σκοπός της παρούσας εργασίας είναι η ανάπτυξη ενός λειτουργικού Πρωτοτύπου Συστήματος Διαχείρισης Κτηνιατρείου (MVP), χρησιμοποιώντας το μοντέλο **Gemini 3 Pro** ως τον κύριο πόρο συγγραφής κώδικα. Μέσα από αυτή τη διαδικασία, επιδιώκεται η αξιολόγηση της ωριμότητας της τεχνολογίας ΤΝ στη Μηχανική Λογισμικού.

Οι επιμέρους στόχοι της μελέτης είναι:

- **Ανάπτυξη λογισμικού:** Η δημιουργία μιας πλήρους εφαρμογής (Full Stack) με σύγχρονες τεχνολογίες (Spring Boot, React, PostgreSQL).
- **Αξιολόγηση ποιότητας:** Η ποσοτική μέτρηση της ποιότητας του παραγόμενου κώδικα μέσω βιομηχανικών προτύπων και εργαλείων στατικής ανάλυσης (SonarQube).
- **Καταγραφή στρατηγικών:** Η ανάλυση και καταγραφή των στρατηγικών Prompt Engineering που απαιτούνται για την καθοδήγηση της ΤΝ σε πολύπλοκα σενάρια ανάπτυξης.

1.3 Ερευνητικά ερωτήματα

Για την επίτευξη των παραπάνω στόχων, η έρευνα δομείται γύρω από τρία κεντρικά ερευνητικά ερωτήματα:

- **RQ1:** Πόσο εφικτή είναι η ανάπτυξη ενός πολύπλοκου πληροφοριακού συστήματος σχεδόν αποκλειστικά με τη χρήση βοηθών Τεχνητής Νοημοσύνης;
- **RQ2:** Ποια είναι η μετρήσιμη ποιότητα του κώδικα (βάσει μετρικών SonarQube όπως Reliability, Security, Maintainability) που παράγεται από τα Μεγάλα Γλωσσικά Μοντέλα;
- **RQ3:** Πώς επηρεάζει η στρατηγική του Prompt Engineering την ορθότητα του αποτελέσματος και πώς διαμορφώνεται ο ρόλος του προγραμματιστή στη νέα αυτή διαδικασία;

1.4 Μεθοδολογική προσέγγιση

Η εργασία ακολουθεί τη μεθοδολογία Design Science Research (DSR), όπως ορίζεται από τους **Hevner et al. (2004)**. Η προσέγγιση αυτή είναι κατάλληλη για την επίλυση πρακτικών προβλημάτων μέσω της δημιουργίας και αξιολόγησης καινοτόμων τεχνουργημάτων.
Στο πλαίσιο αυτό:

- Το **Artifact** είναι το λογισμικό PIMS που αναπτύχθηκε.
- Η **Αξιολόγηση** πραγματοποιείται μέσω πειραματικών μετρήσεων ποιότητας και απόδοσης, καθώς και μέσω της ανάλυσης των ημερολογίων-καταγραφών ανάπτυξης.

1.5 Διάρθρωση της μελέτης

Η διατριβή οργανώνεται σε έξι κεφάλαια:

- Το **Κεφάλαιο 2** παρουσιάζει τη βιβλιογραφική επισκόπηση γύρω από τα Μεγάλα Γλωσσικά Μοντέλα (LLMs) στη μηχανική λογισμικού, τις στρατηγικές Prompt Engineering και τα πρότυπα ποιότητας κώδικα.
- Το **Κεφάλαιο 3** αναλύει τις απαιτήσεις και τη σχεδίαση του συστήματος PIMS.
- Το **Κεφάλαιο 4** περιγράφει αναλυτικά τη μεθοδολογία υλοποίησης, τις προκλήσεις που αντιμετωπίστηκαν και τη συνεργασία με το Gemini 3 Pro.
- Το **Κεφάλαιο 5** παρουσιάζει τα αποτελέσματα της αξιολόγησης (SonarQube, Lighthouse) και αναλύει την παραγωγικότητα της διαδικασίας.
- Το **Κεφάλαιο 6** συνοψίζει τα συμπεράσματα, απαντά στα ερευνητικά ερωτήματα και προτείνει κατευθύνσεις για μελλοντική έρευνα.

2. Βιβλιογραφική επισκόπηση και θεωρητικό υπόβαθρο

Η παρούσα ενότητα επιδιώκει να χαρτογραφήσει το σύγχρονο ερευνητικό τοπίο γύρω από την αξιοποίηση των Μεγάλων Γλωσσικών Μοντέλων (LLMs) στη Μηχανική Λογισμικού. Μέσα από τη συστηματική επισκόπηση της πρόσφατης βιβλιογραφίας, αναλύονται τρεις κεντρικοί άξονες που διέπουν την παρούσα διατριβή: η επίδραση των Μεγάλων Γλωσσικών Μοντέλων (LLMs) στον κύκλο ανάπτυξης λογισμικού, η αναδυόμενη επιστήμη της Μηχανικής Υποβολής Ερωτημάτων (Prompt Engineering) και, τέλος, τα πρότυπα αξιολόγησης της ποιότητας του παραγόμενου κώδικα.

2.1 Τα Μεγάλα Γλωσσικά Μοντέλα (LLMs) στη Μηχανική Λογισμικού

Η ενσωμάτωση των Μεγάλων Γλωσσικών Μοντέλων (LLMs) στον κύκλο ζωής ανάπτυξης λογισμικού (SDLC) αποτελεί, σύμφωνα με την πλειονότητα των ερευνητών, μία από τις σημαντικότερες τεχνολογικές τομές της τελευταίας δεκαετίας. Από τα συμβατικά εργαλεία στατικής ανάλυσης και αυτόματης συμπλήρωσης, η βιομηχανία μεταβαίνει σε μια εποχή όπου οι ευφυείς βοηθοί AI Copilots αναλαμβάνουν ενεργό ρόλο στη σύνθεση, τη βελτιστοποίηση και την τεκμηρίωση του κώδικα.

2.1.1 Εξέλιξη, δυνατότητες και παραγωγικότητα

Η βιβλιογραφία υποδεικνύει ότι τα Μεγάλα Γλωσσικά Μοντέλα (LLMs), όπως το GPT-4 (OpenAI) και το Gemini (Google), δεν λειτουργούν απλώς ως βάσεις γνώσης, αλλά ως μηχανές συλλογισμού που μπορούν να κατανοήσουν σύνθετες απαιτήσεις. Σύμφωνα με την εμπειρική μελέτη του **Guo (2024)**, η οποία εξέτασε την αποτελεσματικότητα διαφορετικών μοντέλων στην παραγωγή κώδικα σε γλώσσες όπως η Python και η Java, διαπιστώνεται ότι η χρήση Μεγάλων Γλωσσικών Μοντέλων (LLMs) επιταχύνει δραματικά τη διαδικασία ανάπτυξης, ιδιαίτερα σε επαναλαμβανόμενες εργασίες ή στη δημιουργία πρότυπου κώδικα. Τα ευρήματα του Guo καταδεικνύουν ότι τα μοντέλα αυτά επιδεικνύουν αξιοσημείωτη ικανότητα στην κατανόηση φυσικής γλώσσας και τη μετατροπή της σε εκτελέσιμη λογική, μειώνοντας το γνωστικό φορτίο του προγραμματιστή, επιτρέποντάς τους να εστιάσουν σε εργασίες υψηλότερης προστιθέμενης αξίας, όπως η αρχιτεκτονική και η επίλυση σύνθετων προβλημάτων.

Στο ίδιο μήκος κύματος, οι **Wang et al. (2024)** υπογραμμίζουν ότι η συνεισφορά των Μεγάλων Γλωσσικών Μοντέλων (LLMs) δεν είναι ομοιόμορφη σε όλα τα επίπεδα ανάπτυξης. Ενώ αριστεύουν στη δημιουργία μεμονωμένων συναρτήσεων ή κλάσεων,

η απόδοσή τους εξαρτάται άμεσα από την πολυπλοκότητα του προβλήματος. Η έρευνά τους εισάγει την έννοια της «Πρόβλεψης Πολυπλοκότητας Κώδικα» (Code Complexity Prediction) ως καθοριστικό παράγοντα για την επιτυχία της παραγωγής κώδικα, υποδεικνύοντας ότι όσο αυξάνεται η κυκλωματική πολυπλοκότητα, τόσο μειώνεται η ακρίβεια του μοντέλου χωρίς την κατάλληλη ανθρώπινη καθοδήγηση.

2.1.2 Κίνδυνοι ασφάλειας και ευπάθειες κώδικα

Παρά τα αδιαμφισβήτητα οφέλη στην παραγωγικότητα, η ενσωμάτωση των LLMs εγείρει σοβαρά ζητήματα ασφάλειας και αξιοπιστίας. Μια κριτική θεώρηση της τεχνολογίας προσφέρεται από τους **Bruni et al. (2025)**, οι οποίοι διενήργησαν μια εκτεταμένη συγκριτική αξιολόγηση εστιάζοντας στην ασφάλεια του παραγόμενου κώδικα.

Τα ευρήματα των Bruni et al. είναι ιδιαίτερα ανησυχητικά για την ανάπτυξη κρίσιμων συστημάτων (όπως το PIMS της παρούσας μελέτης). Η έρευνα κατέδειξε ότι, απουσία ειδικών περιορισμών, τα Μεγάλα Γλωσσικά Μοντέλα (LLMs) τείνουν να παράγουν κώδικα που είναι μεν λειτουργικός, αλλά ευάλωτος σε κοινές επιθέσεις. Συγκεκριμένα, παρατηρήθηκε υψηλή συχνότητα εμφάνισης ευπαθειών όπως SQL Injection (SQLi) και Cross-Site Scripting (XSS), καθώς τα μοντέλα συχνά αναπαράγουν ανασφαλή μοτίβα που εντόπισαν στα δεδομένα εκπαίδευσής τους.

Επιπλέον, οι ερευνητές επισημαίνουν το φαινόμενο της «ψευδαίσθησης ασφάλειας», όπου ο κώδικας φαίνεται συντακτικά άρτιος, αλλά παραβιάζει βασικές αρχές όπως η αρχή των ελάχιστων προνομίων ή η ορθή διαχείριση μυστικών. Συνεπώς, η βιβλιογραφία συγκλίνει στο συμπέρασμα ότι η χρήση Μεγάλων Γλωσσικών Μοντέλων (LLMs) απαιτεί αυστηρή ανθρώπινη επίβλεψη και την εφαρμογή στρατηγικών "Secure-by-Design Prompting" για τον μετριασμό αυτών των κινδύνων.

2.2 Μηχανική Υποβολής Ερωτημάτων (Prompt Engineering) για Προγραμματιστές

Η μετάβαση από τον παραδοσιακό προγραμματισμό στον προγραμματισμό με τη βοήθεια ΤΝ έχει αναδείξει τη Μηχανική Υποβολής Ερωτημάτων (Prompt Engineering) ως μια κρίσιμη δεξιότητα. Η βιβλιογραφία απομακρύνεται πλέον από την αντιμετώπιση του Prompting ως «τέχνη» και το προσεγγίζει ως μια συστηματική μηχανική διαδικασία με συγκεκριμένες τεχνικές και μοτίβα.

2.2.1 Η αναγκαιότητα της δομημένης εκπαίδευσης

Η σημασία της συστηματικής προσέγγισης στο Prompt Engineering αναδεικνύεται από τους **Marvin et al. (2023)**. Στην έρευνά τους σχετικά με την εκπαίδευση στη Μηχανική Λογισμικού, τονίζουν ότι η χρήση εργαλείων όπως το ChatGPT δεν πρέπει να αντιμετωπίζεται ως «μαγική λύση», αλλά ως μια νέα δεξιότητα που απαιτεί κριτική σκέψη. Οι Marvin et al. υποστηρίζουν ότι οι προγραμματιστές πρέπει να

εκπαιδεύονται στο να αναλύουν το πρόβλημα και να το διατυπώνουν με ακρίβεια στις εντολές τους, καθώς η ποιότητα της εξόδου είναι άμεσα συναρτώμενη με την ποιότητα της εισόδου ("Garbage In, Garbage Out").

2.2.2 Δομημένη ανάπτυξη κώδικα και Prompt Construction

Μια σημαντική συμβολή στην τυποποίηση της διαδικασίας προσφέρουν οι **Li, Shi και Zhang (2024)**. Στην εργασία τους για την ταχεία ανάπτυξη κώδικα, προτείνουν μια πλατφόρμα που βασίζεται σε έναν «Κατασκευαστή Prompts» (Prompt Builder). Οι ερευνητές υποστηρίζουν ότι ένα αποτελεσματικό prompt για ανάπτυξη λογισμικού δεν είναι μια απλή πρόταση, αλλά μια σύνθετη δομή που πρέπει να περιλαμβάνει πέντε διακριτά στοιχεία:

- **Ρόλος (Role):** Π.χ. "Act as a Senior Java Developer".
- **Πλαίσιο (Context):** Πληροφορίες για το project και τις τεχνολογίες.
- **Οδηγία (Instruction):** Η συγκεκριμένη λειτουργική απαίτηση.
- **Περιορισμός (Constraint):** Π.χ. "Do not use Lombok", "Use Java 17".
- **Παράδειγμα (Example):** Few-shot δείγματα κώδικα.

Η προσέγγιση των **Li et al. (2024)** επιβεβαιώνει ότι η δυναμική και ολοκληρωμένη κατασκευή prompts βελτιώνει δραματικά την ποιότητα του παραγόμενου κώδικα, μειώνοντας την ανάγκη για πολλαπλές διορθώσεις.

2.2.3 Τεχνικές Prompting και πολυπλοκότητα

Η αποτελεσματικότητα της αλληλεπίδρασης με τα Μεγάλα Γλωσσικά Μοντέλα (LLMs) καθορίζεται σε μεγάλο βαθμό από την επιλογή της κατάλληλης τεχνικής υποβολής ερωτήματος. Οι **Guo (2024)** και **Wang et al. (2024)** αναλύουν διεξοδικά τρεις κύριες προσεγγίσεις:

- **Zero-shot Prompting:** Η απευθείας υποβολή του αιτήματος χωρίς την παροχή παραδειγμάτων. Σύμφωνα με τον **Guo (2024)**, αυτή η μέθοδος είναι επαρκής για απλές εργασίες ρουτίνας (π.χ. δημιουργία ενός DTO), αλλά συχνά αποτυγχάνει σε αλγοριθμικά σύνθετα προβλήματα.
- **Few-shot Prompting:** Η παροχή ζευγών εισόδου-εξόδου ως παραδείγματα εντός του prompt, η οποία βοηθά το μοντέλο να κατανοήσει το αναμενόμενο format και τη λογική.
- **Chain-of-Thought (CoT):** Η τεχνική όπου ζητείται από το μοντέλο να αναλύσει βήμα-προς-βήμα τη σκέψη του. Σύμφωνα με τη βιβλιογραφία, η εφαρμογή του CoT είναι κρίσιμη για την επίλυση προβλημάτων που απαιτούν λογικό συλλογισμό πολλαπλών βημάτων, καθώς επιτρέπει στο μοντέλο να "αυτο-διορθώνεται" κατά τη διαδικασία παραγωγής.

Η συνεισφορά των **Wang et al. (2024)** είναι καθοριστική σε αυτό το σημείο, καθώς συνδέουν την επιλογή της τεχνικής με την πολυπλοκότητα του κώδικα. Υποστηρίζουν ότι για κώδικα υψηλής πολυπλοκότητας, η χρήση Chain-of-Thought είναι

επιβεβλημένη, καθώς μειώνει δραστικά τα λογικά σφάλματα και τις «παραισθήσεις» (hallucinations), επιτρέποντας στο μοντέλο να αποσυνθέσει το πρόβλημα σε διαχειρίσιμα υπο-προβλήματα.

2.2.4 Μοτίβα Prompting (Prompt Patterns)

Κεντρική θέση στο θεωρητικό υπόβαθρο της παρούσας εργασίας κατέχει η πρωτοποριακή εργασία των **White et al. (2024)**. Οι συγγραφείς εισάγουν την έννοια των "Prompt Patterns", κατ' αναλογία με τα καθιερωμένα "Design Patterns" της μηχανικής λογισμικού (όπως αυτά των Gang of Four). Τα μοτίβα αυτά αποτελούν τυποποιημένες, επαναχρησιμοποιήσιμες δομές prompts που επιλύουν συχνά προβλήματα επικοινωνίας και ανάθεσης εργασιών στα Μεγάλα Γλωσσικά Μοντέλα (LLMs).

Μεταξύ των μοτίβων που περιγράφουν οι White et al., ιδιαίτερη βαρύτητα για την ανάπτυξη λογισμικού έχουν:

- **Persona Pattern:** Η ανάθεση συγκεκριμένου ρόλου στην ΤΝ (π.χ. *"Act as a Senior Spring Boot Architect"*). Αυτό το μοτίβο ενεργοποιεί το κατάλληλο σημασιολογικό πλαίσιο, προσαρμόζοντας το λεξιλόγιο και το επίπεδο τεχνικής λεπτομέρειας της απάντησης.
- **Refinement Pattern:** Μια επαναληπτική διαδικασία όπου η ΤΝ καλείται να βελτιώσει τον κώδικα που παρήγαγε, βάσει νέων κριτηρίων ή ανατροφοδότησης (π.χ. *"Refactor this code to remove Lombok annotations"*).
- **Cognitive Verifier Pattern:** Η τεχνική όπου ζητείται από το μοντέλο να θέσει διευκρινιστικές ερωτήσεις στον χρήστη πριν προχωρήσει στην υλοποίηση (π.χ. *"Ask me questions to clarify the requirements before generating code"*). Αυτό το μοτίβο είναι κρίσιμο για την αποφυγή αυθαίρετων παραδοχών εκ μέρους της ΤΝ.

Η εφαρμογή αυτών των μοτίβων, όπως υποστηρίζουν και οι **Li et al. (2024)**, μετατρέπει το Prompting από μια εμπειρική διαδικασία δοκιμής και σφάλματος (trial-and-error) σε μια δομημένη μεθοδολογία, η οποία, όπως θα φανεί στα επόμενα κεφάλαια, αποτέλεσε τον ακρογωνιαίο λίθο για την ανάπτυξη του συστήματος PIMS.

2.3 Ποιότητα λογισμικού και Μετρικές αξιολόγησης

Η ευκολία με την οποία τα Μεγάλα Γλωσσικά Μοντέλα (LLMs) παράγουν κώδικα δημιουργεί την ανάγκη για αυστηρότερα πλαίσια αξιολόγησης της ποιότητας. Ο κώδικας που παράγεται αυτόματα μπορεί να είναι λειτουργικά ορθός, αλλά δομικά ακατάλληλος, δυσχεραίνοντας τη συντήρηση και την επέκταση του συστήματος. Το θεωρητικό υπόβαθρο της αξιολόγησης βασίζεται στο διεθνές πρότυπο **ISO/IEC 25010**, το οποίο ορίζει τα χαρακτηριστικά ποιότητας λογισμικού, συμπεριλαμβανομένης της Λειτουργικής Καταλληλότητας, της Αξιοπιστίας, της Ασφάλειας και της Συντηρησιμότητας.

2.3.1 Η διάσταση της λειτουργικότητας έναντι της δομής

Οι **Liu et al. (2023)** προσφέρουν σημαντική ανάλυση για τη διάκριση μεταξύ της επιφανειακής ομοιότητας και της ουσιαστικής ποιότητας του κώδικα. Στην εργασία τους, ασκούν κριτική στις παραδοσιακές μετρικές αξιολόγησης NLP, όπως το **CodeBLEU**, οι οποίες μετρούν τη λεξιλογική ομοιότητα του παραγόμενου κώδικα με έναν κώδικα αναφοράς. Οι συγγραφείς υποστηρίζουν ότι στην πρακτική μηχανική λογισμικού, αυτές οι μετρικές είναι ανεπαρκείς.

Αντιθέτως, προκρίνουν τη χρήση της μετρικής **Pass@k**, η οποία αξιολογεί τη λειτουργική ορθότητα μέσω της επιτυχούς εκτέλεσης Unit Tests. Ωστόσο, ακόμα και η Pass@k δεν καλύπτει τη δομική υγεία του κώδικα.

2.3.2 Αυτοματοποιημένη παραγωγή μετρικών

Συμπληρωματικά προς την αξιολόγηση, ο **Rosa (2024)** διερεύνησε τη δυνατότητα ενσωμάτωσης του ChatGPT απευθείας σε εργαλεία παραγωγής μετρικών λογισμικού. Η μελέτη του Rosa εξετάζει κατά πόσον τα Μεγάλα Γλωσσικά Μοντέλα (LLMs) μπορούν να αντικαταστήσουν ή να ενισχύσουν τα παραδοσιακά εργαλεία στατικής ανάλυσης στην εξαγωγή μετρικών. Το συμπέρασμά του είναι ότι, ενώ τα Μεγάλα Γλωσσικά Μοντέλα (LLMs) μπορούν να αναλύσουν κώδικα και να προτείνουν βελτιώσεις μετρικών, η διαδικασία απαιτεί προσεκτική επικύρωση, καθώς τα μοντέλα ενδέχεται να παρερμηνεύσουν το πλαίσιο εκτέλεσης.

2.3.3 Στατική ανάλυση και Τεχνικό χρέος

Για την ολιστική αξιολόγηση του AI-generated κώδικα, η βιβλιογραφία στρέφεται σε εργαλεία Στατικής Ανάλυσης Κώδικα (Static Code Analysis). Όπως αναφέρεται σε σχετικές μελέτες (συμπεριλαμβανομένης της ανάλυσης των Wang et al.), κρίσιμες μετρικές είναι:

- **Κυκλωματική Πολυπλοκότητα (Cyclomatic Complexity):** Ο αριθμός των ανεξάρτητων μονοπατιών εκτέλεσης στον κώδικα. Υψηλή πολυπλοκότητα σε κώδικα παραγόμενο από ΤΝ υποδηλώνει ότι το μοντέλο δεν κατάφερε να εφαρμόσει σωστά την αρχή του διαχωρισμού ευθυνών (Separation of Concerns).
- **Code Smells:** Ενδείξεις κακού σχεδιασμού (π.χ. μέθοδοι-μαμούθ, διπλότυπος κώδικας) που αυξάνουν το Τεχνικό χρέος (Technical Debt).
- **Γνωστική Πολυπλοκότητα (Cognitive Complexity):** Μια μέτρηση του πόσο δύσκολο είναι για έναν άνθρωπο να κατανοήσει τη ροή του κώδικα, στοιχείο κρίσιμο όταν ο άνθρωπος καλείται να συντηρήσει κώδικα που έγραψε μια μηχανή.

Συνοψίζοντας, η σύγχρονη βιβλιογραφία συγκλίνει στο ότι τα Μεγάλα Γλωσσικά Μοντέλα (LLMs) αποτελούν ισχυρούς πολλαπλασιαστές ισχύος για τους προγραμματιστές, υπό την προϋπόθεση ότι η χρήση τους πλαισιώνεται από: α) εξελιγμένες στρατηγικές Prompt Engineering (Li et al., 2024, White et al., 2024), β)

επίγνωση των κινδύνων ασφαλείας (Bruni et al., 2025) και γ) αυστηρή μέτρηση της δομικής ποιότητας πέραν της απλής λειτουργικότητας (Liu et al., ISO/IEC 25010). Αυτό το τρίπτυχο αποτελεί το θεωρητικό θεμέλιο πάνω στο οποίο βασίζεται η μεθοδολογία και η αξιολόγηση της παρούσας διπλωματικής εργασίας.

3. Ανάλυση και σχεδίαση συστήματος

3.1 Ανάλυση επιχειρησιακών αναγκών

Η άσκηση της σύγχρονης κτηνιατρικής ιατρικής απαιτεί την ταχεία και αλάνθαστη διαχείριση ενός μεγάλου όγκου κλινικών δεδομένων. Η επιχειρηματική ανάγκη για την ανάπτυξη του «Ολοκληρωμένου Συστήματος Διαχείρισης Κτηνιατρείου» (Practice Information Management System - PIMS) πηγάζει από την ανεπάρκεια των συμβατικών, συχνά χειρόγραφων μεθόδων τήρησης αρχείου, να υποστηρίξουν αποτελεσματικά την κλινική αριστεία και την ασφάλεια των ασθενών.

Στην τρέχουσα κατάσταση (AS-IS), η χρήση φυσικών φακέλων και αποσπασματικών σημειώσεων οδηγεί σε σοβαρά λειτουργικά προβλήματα. Η διασπορά πληροφοριών μεταξύ διαφορετικών τετραδίων ραντεβού και ιστορικού καθιστά δυσχερή την ανάκτηση της πλήρους κλινικής εικόνας ενός ζώου σε πραγματικό χρόνο. Αυτό εγκυμονεί κινδύνους ιατρικών λαθών, όπως η χορήγηση φαρμάκων σε αλλεργικούς ασθενείς ή η παράλειψη κρίσιμων επανεξετάσεων.

Συνεπώς, η κύρια επιχειρησιακή απαίτηση είναι η μετάβαση σε ένα ψηφιακό περιβάλλον που εστιάζει στην κεντρικοποίηση της κλινικής πληροφορίας. Το σύστημα καλείται να διασφαλίσει την ακεραιότητα του ιατρικού ιστορικού, την τυποποίηση της κλινικής καταγραφής και την άμεση προσβασιμότητα στα δεδομένα των ασθενών από το εξουσιοδοτημένο ιατρικό προσωπικό, βελτιστοποιώντας έτσι την ποιότητα της παρεχόμενης φροντίδας.

3.2 Λειτουργικές απαιτήσεις συστήματος

Το σύστημα σχεδιάστηκε για να καλύψει τον πυρήνα των κλινικών λειτουργιών. Η ανάλυση των απαιτήσεων οδήγησε στον καθορισμό των παρακάτω υποσυστημάτων:

3.2.1 Υποσύστημα διαχείρισης πελατών και ασθενών

Ο ακρογωνιαίος λίθος της εφαρμογής είναι η διαχείριση της βάσης δεδομένων κηδεμόνων (Owners) και ασθενών (Pets). Το σύστημα υιοθετεί μια ιεραρχική δομή, όπου κάθε κηδεμόνας δύναται να συνδέεται με πολλαπλές οντότητες ζώων.

- **Διαχείριση κηδεμόνα:** Υποστηρίζεται η καταγραφή και επεξεργασία των στοιχείων επικοινωνίας (Ονοματεπώνυμο, Τηλέφωνο, Email), επιτρέποντας την

ταχεία ταυτοποίηση.

- **Διαχείριση ασθενούς:** Για κάθε ζώο τηρείται ψηφιακή καρτέλα με αναλυτικά στοιχεία: Όνομα, Είδος, Φυλή, Ημερομηνία γέννησης, Φύλο και αριθμό Microchip.

3.2.2 Υποσύστημα διαχείρισης ραντεβού

Η οργάνωση του κλινικού χρόνου επιτυγχάνεται μέσω ενός ψηφιακού ημερολογίου. Το σύστημα επιτρέπει τη δημιουργία, τροποποίηση και ακύρωση ραντεβού. Κάθε εγγραφή ραντεβού διασυνδέει τρεις οντότητες: τον Ασθενή, τον Κτηνίατρο και τον Χρόνο (Ημερομηνία/Ώρα).

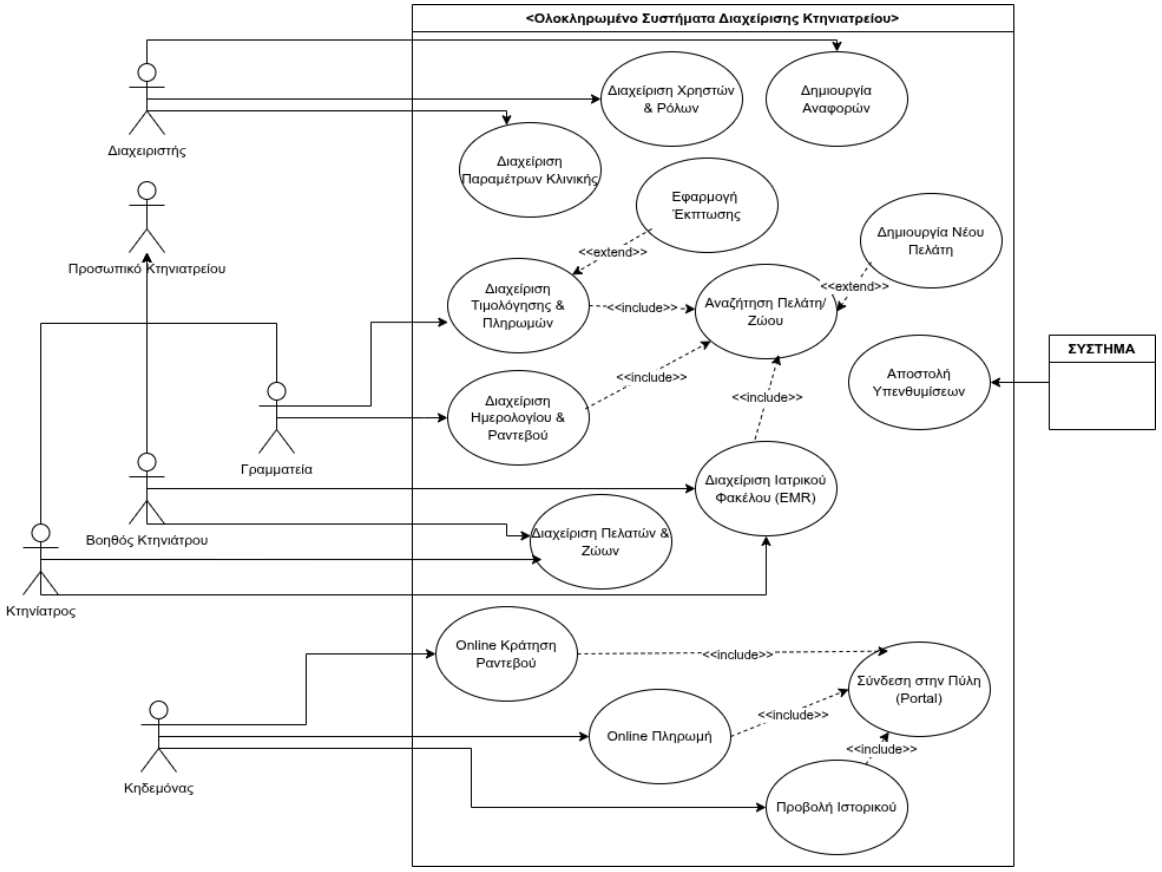
Υποστηρίζεται η κατηγοριοποίηση των ραντεβού βάσει του λόγου επίσκεψης (π.χ. Εμβολιασμός, Παθολογικό, Χειρουργείο), γεγονός που βοηθά το προσωπικό στην προετοιμασία των απαραίτητων πόρων πριν την άφιξη του ασθενούς.

3.2.3 Ηλεκτρονικός Ιατρικός Φάκελος (EMR & SOAP)

Το υποσύστημα του Ηλεκτρονικού Ιατρικού Φακέλου αποτελεί το κεντρικό σημείο της κλινικής λειτουργίας. Για τη διασφάλιση της ποιότητας και της συνέπειας των δεδομένων, το σύστημα επιβάλλει τη χρήση του προτύπου **SOAP** (Subjective, Objective, Assessment, Plan) για κάθε επίσκεψη. Η δομή της πληροφορίας αναλύεται ως εξής:

- **Subjective (Υποκειμενικά):** Πεδίο κειμένου για την καταγραφή του ιστορικού, των παραπόνων του κηδεμόνα και της περιγραφής των συμπτωμάτων.
- **Objective (Αντικειμενικά):** Καταγραφή των μετρήσιμων ευρημάτων της κλινικής εξέτασης. Το σύστημα παρέχει δομημένα πεδία για τα ζωτικά σημεία (Θερμοκρασία, Βάρος) και πεδίο κειμένου για τα ευρήματα της ψηλάφησης και της ακρόασης.
- **Assessment (Αξιολόγηση):** Καταγραφή της ιατρικής κρίσης, της διαφορικής διάγνωσης και της τελικής διάγνωσης.
- **Plan (Σχέδιο):** Πεδίο καταγραφής των θεραπευτικών οδηγιών, της συνταγογράφησης φαρμάκων και του προγραμματισμού επανεξετάσεων.

Κάθε ιατρική εγγραφή (Visit) συνδέεται άρρηκτα με τον Ασθενή και φέρει χρονοσήμανση, δημιουργώντας ένα αμετάβλητο χρονολόγιο του ιστορικού υγείας του ζώου.



Εικόνα 3.1: Αρχικό διάγραμμα περιπτώσεων χρήσης

3.3 Μη λειτουργικές απαιτήσεις

Ο σχεδιασμός του συστήματος έλαβε υπόψη κρίσιμες μη λειτουργικές παραμέτρους για τη διασφάλιση της ορθής λειτουργίας:

- **Ασφάλεια δεδομένων (Data Security):** Εφαρμογή μηχανισμών αυθεντικοποίησης χρηστών και κρυπτογράφησης των κωδικών πρόσβασης στη βάση δεδομένων.
- **Ακεραιότητα δεδομένων (Data Integrity):** Διασφάλιση της συνοχής των δεδομένων μέσω περιορισμών ξένων κλειδιών στη βάση, ώστε να μην είναι δυνατή η ύπαρξη «ορφανών» ιατρικών εγγραφών χωρίς σύνδεση με ασθενή.
- **Απόδοση (Performance):** Βελτιστοποίηση των ερωτημάτων στη βάση δεδομένων και χρήση ασύγχρονης επικοινωνίας στο frontend για άμεση απόκριση του διεπαφής χρήστη.
- **Επεκτασιμότητα (Scalability):** Η αρχιτεκτονική σχεδιάστηκε ώστε να επιτρέπει τη μελλοντική προσθήκη νέων modules χωρίς την ανάγκη ανακατασκευής του πυρήνα.

3.4 Αρχιτεκτονική και Τεχνολογίες

Η υλοποίηση του συστήματος βασίστηκε σε μια πολυεπίπεδη αρχιτεκτονική (Multi-tier Architecture), διαχωρίζοντας σαφώς το επίπεδο παρουσίασης (Frontend), το επίπεδο επιχειρησιακής λογικής (Backend) και το επίπεδο δεδομένων (Database).

3.4.1 Backend: Spring Boot & Ροή Δεδομένων

Το Backend αναπτύχθηκε με το framework **Spring Boot (Java)**, ακολουθώντας την αρχιτεκτονική MVC (Model-View-Controller) προσαρμοσμένη για RESTful APIs. Η ροή της πληροφορίας ακολουθεί αυστηρά τα παρακάτω επίπεδα:

1. **REST Controller Layer:** Αποτελεί το σημείο εισόδου (Entry Point). Οι Controllers δέχονται τα HTTP Requests (GET, POST, PUT, DELETE) από το Frontend. Μετατρέπουν το εισερχόμενο JSON σώμα (Request Body) σε αντικείμενα Java (DTOs - Data Transfer Objects) και αναθέτουν το αίτημα στο κατάλληλο Service.
2. **Service Layer:** Εδώ υλοποιείται η επιχειρησιακή λογική. Τα Services περιέχουν τους κανόνες του συστήματος. Για παράδειγμα, κατά τη δημιουργία μιας επίσκεψης, το Service επαληθεύει ότι ο ασθενής υπάρχει και είναι ενεργός πριν προχωρήσει στην αποθήκευση. Χρησιμοποιείται το Dependency Injection (@Autowired) για τη διασύνδεση των εξαρτήσεων.
3. **Repository Layer (Data Access):** Χρησιμοποιείται το **Spring Data JPA**. Τα Repositories επεκτείνουν το JpaRepository και παρέχουν αφαιρετικές μεθόδους για την εκτέλεση CRUD πράξεων στη βάση, χωρίς την ανάγκη συγγραφής ακατέργαστης SQL.

3.4.1.1 Δομή και οργάνωση πηγαίου κώδικα (Project Structure)

Η οργάνωση του πηγαίου κώδικα του Backend ακολουθεί αυστηρά την αρχή του **Διαχωρισμού των Αρμοδιοτήτων (Separation of Concerns)**, υιοθετώντας την τυποποιημένη διάρθρωση πακέτων του Spring Boot. Η συγκεκριμένη δομή διασφαλίζει τη συντηρησιμότητα, την επεκτασιμότητα και τη σαφή οριοθέτηση των επιπέδων της εφαρμογής, όπως αποτυπώνεται στο δέντρο φακέλων του έργου.

Στη βάση της ιεραρχίας βρίσκεται το πακέτο **config**, το οποίο περιλαμβάνει τις θεμελιώδεις ρυθμίσεις της εφαρμογής, όπως η διαμόρφωση της ασφάλειας (SecurityConfig) και η αρχικοποίηση δεδομένων (DataInitializer). Σε συνδυασμό με το πακέτο **security**, όπου υλοποιούνται οι μηχανισμοί πιστοποίησης (π.χ. JwtAuthenticationFilter, JwtService), συγκροτείται η υποδομή για την stateless αυθεντικοποίηση μέσω JSON Web Tokens (JWT), θωρακίζοντας τα endpoints από μη εξουσιοδοτημένη πρόσβαση.

Ο πυρήνας του μοντέλου δεδομένων ορίζεται στο πακέτο **entity**, το οποίο περιέχει τις κλάσεις που αντιστοιχίζονται στους πίνακες της βάσης δεδομένων (ORM Entities), όπως Patient, Appointment και MedicalRecord. Η ακεραιότητα και η τυποποίηση των τιμών των πεδίων ενισχύεται μέσω του πακέτου **enums**, όπου

ορίζονται σταθερές απαριθμήσεις για κρίσιμες μεταβλητές (π.χ. Species, AppointmentStatus), αποτρέποντας την εισαγωγή αυθαίρετων τιμών.

Κρίσιμο ρόλο στην αρχιτεκτονική διαδραματίζει το πακέτο **dto (Data Transfer Objects)**. Η ύπαρξη ξεχωριστών κλάσεων για τη μεταφορά δεδομένων (π.χ. PatientRequest, AuthResponse) αποδεικνύει την αποσύζευξη του εσωτερικού μοντέλου της βάσης από το δημόσιο API. Μέσω των DTOs, επιτυγχάνεται η απόκρυψη ευαίσθητων πεδίων των Entities και η διαμόρφωση ενός αυστηρού συμβολαίου επικοινωνίας με το Frontend, προστατεύοντας το σύστημα από ακούσιες τροποποιήσεις δεδομένων.

Η ροή εκτέλεσης ακολουθεί την κλασική αρχιτεκτονική τριών επιπέδων:

- **Controller Layer (controller):** Λειτουργεί ως το σημείο εισόδου των HTTP αιτημάτων. Οι Controllers (π.χ. AppointmentController) δεν περιέχουν επιχειρησιακή λογική, αλλά ενορχηστρώνουν τη ροή, καλώντας τις κατάλληλες υπηρεσίες.
- **Service Layer (service & service/impl):** Εδώ εντοπίζεται η επιχειρησιακή λογική. Η χρήση διεπαφών (interfaces) στο πακέτο service και των υλοποιήσεών τους στο service/impl (π.χ. AppointmentServiceImpl) προάγει τη χαλαρή σύζευξη (loose coupling), επιτρέποντας την εύκολη εναλλαγή υλοποιήσεων ή τη δημιουργία mocks για Unit Testing.
- **Data Access Layer (repository):** Το πακέτο repository παρέχει το επίπεδο αφαίρεσης για την πρόσβαση στα δεδομένα, επεκτείνοντας τις διεπαφές του Spring Data JPA και επιτρέποντας την εκτέλεση ερωτημάτων στη βάση χωρίς την ανάγκη συγγραφής κώδικα SQL.

```
iosifidis@ubuntu:~/Documents/git/UoM/msc-pims/backend/src/main/java/com/pims/backend(mvp3)$ tree
├── BackendApplication.java
├── config
│   ├── ApplicationConfig.java
│   ├── DataInitializer.java
│   ├── JacksonConfig.java
│   ├── SecurityConfig.java
│   └── WebConfig.java
├── controller
│   ├── AppointmentController.java
│   ├── AuthController.java
│   ├── ClientController.java
│   ├── DashboardController.java
│   ├── MedicalRecordController.java
│   ├── PatientController.java
│   └── UserController.java
├── dto
│   ├── AppointmentRequest.java
│   ├── auth
│   │   ├── AuthResponse.java
│   │   ├── LoginRequest.java
│   │   └── RegisterRequest.java
│   ├── ClientRequest.java
│   ├── DashboardStats.java
│   ├── MedicalRecordRequest.java
│   └── PatientRequest.java
├── entity
│   ├── Appointment.java
│   ├── AppUser.java
│   ├── Client.java
│   ├── MedicalRecord.java
│   ├── Patient.java
│   ├── Resource.java
│   └── Role.java
├── enums
│   ├── AppointmentStatus.java
│   ├── AppointmentType.java
│   ├── ResourceType.java
│   ├── Sex.java
│   └── Species.java
├── repository
│   ├── AppointmentRepository.java
│   ├── AppUserRepository.java
│   ├── ClientRepository.java
│   ├── MedicalRecordRepository.java
│   ├── PatientRepository.java
│   ├── ResourceRepository.java
│   └── RoleRepository.java
├── security
│   ├── JwtAuthenticationEntryPoint.java
│   ├── JwtAuthenticationFilter.java
│   └── JwtService.java
├── service
│   ├── AppointmentService.java
│   ├── ClientService.java
│   ├── impl
│   │   ├── AppointmentServiceImpl.java
│   │   ├── ClientServiceImpl.java
│   │   ├── MedicalRecordServiceImpl.java
│   │   ├── PatientServiceImpl.java
│   │   └── UserServiceImpl.java
│   ├── MedicalRecordService.java
│   ├── PatientService.java
│   └── UserService.java

11 directories, 53 files
```

Εικόνα 3.2: Δομή καταλόγων Backend εφαρμογής (Project Structure)

3.4.2 Frontend: React.js

Το επίπεδο παρουσίασης υλοποιήθηκε με τη βιβλιοθήκη **React.js**. Η εφαρμογή λειτουργεί ως Single Page Application (SPA), προσφέροντας πλούσια εμπειρία χρήστη.

- **State Management:** Η διαχείριση της κατάστασης της εφαρμογής γίνεται μέσω των React Hooks (useState, useEffect) και του Context API για την κοινή χρήση δεδομένων (π.χ. στοιχεία συνδεδεμένου χρήστη) σε όλο το δέντρο των components.
- **API Integration:** Η επικοινωνία με το Backend γίνεται ασύγχρονα μέσω της βιβλιοθήκης **Axios**. Τα δεδομένα ανταλλάσσονται σε μορφή JSON. Το Frontend αναλαμβάνει την οπτικοποίηση των δεδομένων και την πρώτη επικύρωση των φορμών εισαγωγής.

3.4.2.1 Δομή και οργάνωση εφαρμογής React (Frontend Structure)

Η αρχιτεκτονική της εφαρμογής διεπαφής (Frontend) ακολουθεί μια αυστηρά αρθρωτή προσέγγιση (modular component-based architecture), σχεδιασμένη για να μεγιστοποιεί την επαναχρησιμοποίηση κώδικα και να διευκολύνει τη μελλοντική επέκταση. Βασικός πυλώνας της οργάνωσης είναι ο σαφής λειτουργικός διαχωρισμός μεταξύ των **Προβολών (pages)** και των **Συστατικών (components)**. Στο φάκελο pages φιλοξενούνται οι πλήρεις σελίδες της εφαρμογής (όπως DashboardPage, PatientsPage), οι οποίες αντιστοιχίζονται άμεσα σε διαδρομές πλοήγησης (routes) και λειτουργούν ως ενορχηστρωτές της διεπαφής. Αντιθέτως, ο φάκελος components περιέχει τα δομικά στοιχεία της εφαρμογής (π.χ. MedicalRecordForm, CalendarView), τα οποία είναι αποσυνδεδεμένα από τη λογική δρμολόγησης και σχεδιασμένα να είναι επαναχρησιμοποιήσιμα σε διαφορετικά σημεία του συστήματος. Αυτή η διάκριση ενισχύει τη συντηρησιμότητα, καθώς επιτρέπει την απομόνωση της λογικής του UI από τη δομή πλοήγησης.

Σε επίπεδο επιχειρησιακής λογικής και διαχείρισης κατάστασης, η εφαρμογή αξιοποιεί το Context API της React για τη διαχείριση της καθολικής κατάστασης (Global State), αποφεύγοντας την πολυπλοκότητα εξωτερικών βιβλιοθηκών. Συγκεκριμένα, το αρχείο context/AuthContext.jsx λειτουργεί ως κεντρικός διαχειριστής της αυθεντικοποίησης, διατηρώντας την πληροφορία του συνδεδεμένου χρήστη προσβάσιμη σε όλο το δέντρο της εφαρμογής. Η ασφάλεια της πλοήγησης ενισχύεται περαιτέρω μέσω του συστατικού ProtectedRoute.jsx (High-Order Component pattern), το οποίο περιβάλλει τις προστατευμένες διαδρομές και αποτρέπει την πρόσβαση σε μη εξουσιοδοτημένους χρήστες. Τέλος, η επικοινωνία με το Backend τυποποιείται μέσω του αρχείου context/axiosConfig.js. Το αρχείο αυτό υλοποιεί το πρότυπο Interceptor, κεντρικοποιώντας τη ρύθμιση των HTTP αιτημάτων (όπως η αυτόματη επισύναψη του JWT token) και τη διαχείριση σφαλμάτων δικτύου, εξασφαλίζοντας συνέπεια και ευκολία στη συντήρηση του κώδικα επικοινωνίας.

```
iosifidis@ubuntu:~/Documents/git/UoM/msc-pims/frontend/src(mvp3)$ tree
├── App.jsx
├── assets
│   └── react.svg
├── components
│   ├── ClientSearchDropdown.jsx
│   ├── MainLayout.jsx
│   ├── MedicalRecordModal.jsx
│   ├── PatientHistory.jsx
│   ├── PatientHistoryModal.jsx
│   └── ProtectedRoute.jsx
├── context
│   ├── AuthContext.jsx
│   └── axiosConfig.js
├── index.css
├── main.jsx
├── pages
│   ├── AppointmentsPage.jsx
│   ├── ClientsPage.jsx
│   ├── DashboardPage.jsx
│   ├── LoginPage.jsx
│   ├── PatientsPage.jsx
│   ├── ProfilePage.jsx
│   ├── RegisterPage.jsx
│   └── UsersPage.jsx

5 directories, 20 files
```

Εικόνα 3.3: Δομή καταλόγων Frontend Εφαρμογής (Project Structure)

3.4.3 Βάση δεδομένων: PostgreSQL

Επιλέχθηκε η **PostgreSQL** ως Σύστημα Διαχείρισης Βάσης Δεδομένων (ΣΔΒΔ). Η επιλογή αυτή στηρίζει την ανάγκη για ισχυρή συσχέτιση δεδομένων και συμμόρφωση με το πρότυπο ACID, στοιχεία κρίσιμα για ιατρικές εφαρμογές.

3.4.4 Σχεδιασμός API και ροή δεδομένων (Data Flow)

Η επικοινωνία μεταξύ του Επιπέδου Παρουσίασης (Frontend Application) και του Επιπέδου Επιχειρησιακής Λογικής (Backend) βασίζεται στο πρότυπο REST (Representational State Transfer). Το API σχεδιάστηκε ώστε να είναι stateless, διασφαλίζοντας την επεκτασιμότητα και την ανεξαρτησία των δύο μερών.

Δεδομένου ότι οι μοναδικοί χρήστες του συστήματος είναι το εξουσιοδοτημένο προσωπικό του κτηνιατρείου (Κτηνίατροι και Διαχειριστές), η ροή της πληροφορίας ακολουθεί μια αυστηρά ελεγχόμενη διαδικασία αυθεντικοποίησης και εξυπηρέτησης αιτημάτων. Η τυπική ροή δεδομένων για την ανάκτηση κλινικών πληροφοριών (π.χ.

προβολή ενός ραντεβού ή ιστορικού ασθενούς) περιγράφεται στα ακόλουθα βήματα:

- **Έναρξη αιτήματος (Frontend Initiation):** Ο κτηνίατρος, αλληλεπιδρώντας με τη διεπαφή χρήστη της React (π.χ. επιλέγοντας ένα ραντεβού από το ημερολόγιο), εκκινεί μια ενέργεια. Η Εφαρμογή Frontend προετοιμάζει ένα HTTP Request (π.χ. GET /api/appointments/{id}).
- **Υποδοχή αιτήματος (Controller Layer):** Το αίτημα παραλαμβάνεται από τον αντίστοιχο **RestController** του Spring Boot (π.χ. AppointmentController). Ο Controller λειτουργεί ως τροχονόμος: δεν εκτελεί επιχειρησιακή λογική, αλλά επικυρώνει τη δομή του αιτήματος και το προωθεί στο κατάλληλο Service.
- **Επεξεργασία λογικής (Service Layer):** Το **Service** (π.χ. AppointmentService) αναλαμβάνει την εκτέλεση της επιχειρησιακής λογικής. Σε αυτό το στάδιο ελέγχονται οι επιχειρησιακοί κανόνες (π.χ. αν το ραντεβού υπάρχει και αν ο χρήστης έχει δικαίωμα πρόσβασης σε αυτό).
- **Ανάκτηση δεδομένων (Repository Layer):** Εφόσον πληρούνται οι προϋποθέσεις, το Service καλεί το **Repository** (π.χ. AppointmentRepository). Μέσω του Spring Data JPA, το Repository εκτελεί το βελτιστοποιημένο SQL ερώτημα στη βάση δεδομένων (PostgreSQL).
- **Σειριοποίηση και απάντηση (DTO Mapping & Response):** Η βάση δεδομένων επιστρέφει τις οντότητες (Entities). Για λόγους ασφαλείας και απόδοσης, το Backend δεν επιστρέφει απευθείας τα Entities (τα οποία αντικατοπτρίζουν τη δομή της βάσης). Αντ' αυτού, τα δεδομένα μετασχηματίζονται σε **Αντικείμενα Μεταφοράς Δεδομένων (Data Transfer Objects - DTOs)**. Τελικώς, το DTO σειριοποιείται σε μορφή JSON και αποστέλλεται ως HTTP Response πίσω στην Εφαρμογή Frontend, η οποία αναλαμβάνει την οπτικοποίηση της πληροφορίας στον Κτηνίατρο.

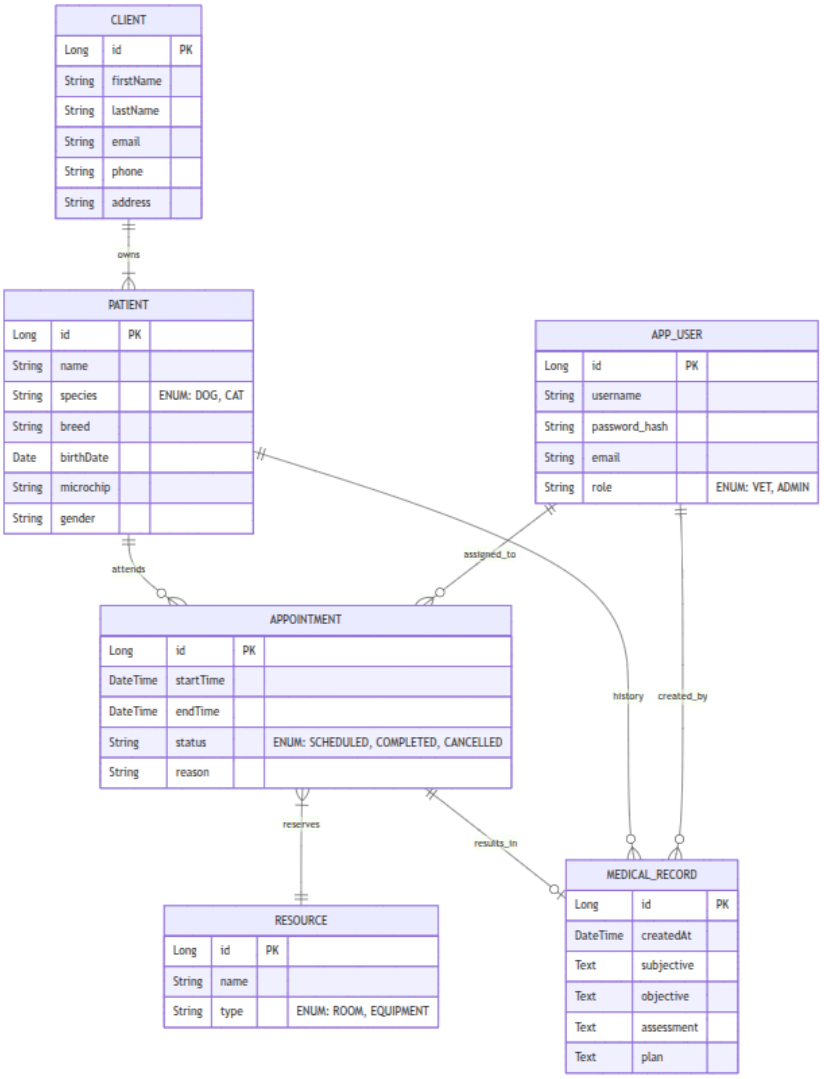
3.5 Σχεδιασμός δεδομένων

Ο σχεδιασμός της βάσης δεδομένων εστιάζει στην ακεραιότητα και τη συνοχή των κλινικών δεδομένων. Το σχήμα της PostgreSQL περιλαμβάνει πίνακες που εξυπηρετούν αποκλειστικά τις ανάγκες του ιατρικού προσωπικού και τη διαχείριση των ασθενών.

Βασική αρχιτεκτονική απόφαση αποτέλεσε ο σαφής διαχωρισμός μεταξύ των **Χρηστών του Συστήματος** (Users) και των **Ιδιοκτητών Ζώων** (Owners).

Στην **Εικόνα 3.3** αποτυπώνεται το Διάγραμμα Οντοτήτων-Συσχετίσεων (ERD) του συστήματος, το οποίο αντικατοπτρίζει τη δομή του Κλινικού Πυρήνα της εφαρμογής. Όπως διακρίνεται, η οντότητα **Client** (Κηδεμόνας) συνδέεται μέσω σχέσης «ένα-προς-πολλά» με την οντότητα **Patient** (Ασθενής), η οποία αποτελεί τον κεντρικό κόμβο του σχήματος, **Appointments** (Ραντεβού) και **MedicalRecords** (Ιατρικό Ιστορικό). Για λόγους ασφάλειας και ιχνηλασιμότητας ενεργειών, ο **AppUser** (Κτηνίατρος) συσχετίζεται άμεσα τόσο με τα προγραμματισμένα ραντεβού όσο και

με τις δημιουργηθείσες ιατρικές εγγραφές, ενώ η ροή της κλινικής πληροφορίας ολοκληρώνεται με τη σύνδεση μεταξύ **Appointment** και **MedicalRecord**, υποδηλώνοντας ότι κάθε ολοκληρωμένη επίσκεψη καταλήγει στη δημιουργία μιας αντίστοιχης εγγραφής στο ιστορικό του ζώου.



Εικόνα 3.4: Διάγραμμα Οντοτήτων-Συσχετίσεων (ERD) της Βάσης Δεδομένων

3.5.1 Πίνακας Users (Προσωπικό Κλινικής)

Ο πίνακας users αντιπροσωπεύει τους ενεργούς δρώντες του συστήματος. Αφορά αποκλειστικά το προσωπικό (Κτηνιάτρους, Διαχειριστές) που έχει δικαίωμα εισόδου (login) στην εφαρμογή.

- **Πεδία:** id (PK), username, password_hash (κρυπτογραφημένος κωδικός), email, role (ENUM: 'VET', 'ADMIN'), active (boolean).
- **Λειτουργία:** Χρησιμοποιείται από το Spring Security για την αυθεντικοποίηση (Authentication) και την εξουσιοδότηση (Authorization).

3.5.2 Πίνακας Owners (Πελάτες / Κηδεμόνες)

Ο πίνακας owners λειτουργεί ως παθητικό μητρώο καταγραφής των κηδεμόνων των ζώων.

- **Σημαντική Διάκριση:** Οι εγγραφές σε αυτόν τον πίνακα **δεν** αντιστοιχούν σε λογαριασμούς χρηστών. Οι ιδιοκτήτες δεν έχουν πρόσβαση στο σύστημα, δεν διαθέτουν κωδικούς εισόδου και δεν αλληλεπιδρούν με την εφαρμογή. Ο πίνακας υπάρχει αποκλειστικά για να συσχετίζει τα ζώα με τα στοιχεία επικοινωνίας των κηδεμόνων τους, εξυπηρετώντας διοικητικές ανάγκες της κλινικής.
- **Πεδία:** id (PK), first_name, last_name, phone_number, email.

3.5.3 Πίνακας Pets (Ασθενείς)

Ο πίνακας pets αποτελεί την κεντρική οντότητα του κλινικού μοντέλου.

- **Συσχέτιση:** Συνδέεται με τον πίνακα owners μέσω σχέσης **Many-to-One** (Πολλά ζώα ανήκουν σε έναν ιδιοκτήτη), διασφαλίζοντας την ακεραιότητα αναφοράς μέσω Foreign Key (owner_id).
- **Πεδία:** id (PK), name, species (π.χ. Dog, Cat), breed, gender, birth_date, microchip_number.

3.5.4 Πίνακας Visits (Ιατρικές Επισκέψεις - EMR)

Ο πίνακας visits αποθηκεύει το ιστορικό των ιατρικών πράξεων και αποτελεί την ψηφιακή αποτύπωση του Ιατρικού Φακέλου.

- **Συσχέτιση:** Κάθε επίσκεψη συνδέεται υποχρεωτικά με ένα ζώο (pet_id) μέσω σχέσης **Many-to-One**.
- **Δομή SOAP:** Ο πίνακας περιλαμβάνει διακριτά πεδία κειμένου (TEXT) για την υποστήριξη του προτύπου SOAP:
  - subjective: Ιστορικό/Συμπτώματα.
  - objective: Κλινικά ευρήματα.
  - assessment: Διάγνωση.
  - plan: Θεραπεία/Συνταγή.
- **Μεταδεδομένα:** Επιπλέον, καταγράφεται η ημερομηνία (visit_date) και, προαιρετικά, ο κτηνίατρος που διενήργησε την εξέταση, συνδέοντας την επίσκεψη με τον πίνακα users.

3.6 Οριοθέτηση πεδίου πρωτοτύπου (Scope Definition)

Στο πλαίσιο της παρούσας διπλωματικής εργασίας, η ανάπτυξη του συστήματος οριοθετήθηκε στρατηγικά ώστε να εστιάσει αποκλειστικά στον **Κλινικό Πυρήνα (Core Clinical Operations)**.

Παρόλο που ένα πλήρες εμπορικό Σύστημα Διαχείρισης Κτηνιατρείου (PIMS) απαιτεί

επιπρόσθετα υποσυστήματα, όπως η Οικονομική Διαχείριση (Τιμολόγηση, Αποθήκη) και η Πύλη Πελατών (Client Portal), αυτά εξαιρέθηκαν σκόπιμα από το πεδίο υλοποίησης του παρόντος πρωτοτύπου.

Η απόφαση αυτή ελήφθη με βάση τη μεθοδολογία της Έρευνας Σχεδιασμού (Design Science Research), προκειμένου να δοθεί η μέγιστη έμφαση στην αξιολόγηση της **Ποιότητας Κώδικα** που παράγεται από τους βοηθούς Τεχνητής Νοημοσύνης σε σύνθετες, κρίσιμες λειτουργίες. Η υλοποίηση του Κλινικού Φακέλου (SOAP) και της διαχείρισης ασθενών παρουσιάζει υψηλότερη λογική πολυπλοκότητα και απαιτήσεις ακεραιότητας δεδομένων σε σύγκριση με απλές CRUD λειτουργίες, καθιστώντας τα ιδανικά πεδία για τη δοκιμασία των δυνατοτήτων του AI Code Generation. Συνεπώς, το παραγόμενο σύστημα αποτελεί ένα λειτουργικό πρωτότυπο κλινικής διαχείρισης (Clinical Management Prototype) και όχι ένα πλήρες προϊόν ERP.

4. Μεθοδολογία υλοποίησης και διαδικασία ανάπτυξης

Η υλοποίηση του πληροφοριακού συστήματος βασίστηκε σε μια επαναληπτική (iterative) διαδικασία ανάπτυξης, όπου ο βοηθός Τεχνητής Νοημοσύνης (Gemini 3 Pro) λειτούργησε ως «Συγκυβερνήτης», αναλαμβάνοντας την παραγωγή κώδικα, ενώ ο άνθρωπος διατήρησε τον ρόλο του αρχιτέκτονα λογισμικού και του ελεγκτή ποιότητας. Το παρόν κεφάλαιο καταγράφει τα τεχνικά μέσα που χρησιμοποιήθηκαν, τη διαδικασία εκκίνησης του έργου, καθώς και τις σημαντικές προκλήσεις που αντιμετωπίστηκαν, με έμφαση στη στρατηγική διαχείριση των βιβλιοθηκών τρίτων.

4.1 Περιβάλλον ανάπτυξης & Εργαλεία

Για την ανάπτυξη του συστήματος επιλέχθηκε ένα σύγχρονο και ελαφρύ περιβάλλον εργασίας, προσανατολισμένο στην υποστήριξη μικρουπηρεσιών και cloud-native εφαρμογών. Ως κύριο Ολοκληρωμένο Περιβάλλον Ανάπτυξης (IDE) χρησιμοποιήθηκε το **Visual Studio Code (VS Code)**. Η ροή εργασίας οργανώθηκε με τη χρήση δύο διακριτών παραθύρων του IDE: ένα αποκλειστικά για την ανάπτυξη του Backend (Spring Boot) και ένα για το Frontend (React). Αυτός ο διαχωρισμός διευκόλυνε την ταυτόχρονη παρακολούθηση των διεργασιών και την απομόνωση των προβλημάτων (Separation of Concerns) σε επίπεδο περιβάλλοντος ανάπτυξης.

Η ενσωμάτωση της Τεχνητής Νοημοσύνης στη ροή εργασίας επιτεύχθηκε μέσω της επέκτασης **Google Cloud Code**. Η επέκταση αυτή λειτούργησε ως η γέφυρα επικοινωνίας με το μοντέλο **Gemini 3 Pro**, επιτρέποντας την υποβολή ερωτημάτων (prompts) και την παραγωγή κώδικα απευθείας εντός του editor, χωρίς την ανάγκη εναλλαγής παραθύρων (context switching).

Για την υποστήριξη της τεχνολογικής στοίβας εγκαταστάθηκαν οι απαραίτητες επεκτάσεις:

- **Extension Pack for Java & Spring Boot Extension Pack:** Για την υποστήριξη της γλώσσας Java 21 και του πλαισίου Spring Boot.
- **ES7+ React/Redux/React-Native snippets:** Για την επιτάχυνση της συγγραφής κώδικα στο Frontend.

Σε επίπεδο υποδομής δεδομένων, υιοθετήθηκε η προσέγγιση της εικονικοποίησης (containerization) μέσω του **Docker Compose**. Η βάση δεδομένων PostgreSQL αναπτύχθηκε ως container, χρησιμοποιώντας την εικόνα postgres:16-alpine, εξασφαλίζοντας ένα απομονωμένο και αναπαραγώγιμο περιβάλλον εκτέλεσης, ανεξάρτητο από το λειτουργικό σύστημα του μηχανήματος ανάπτυξης.

Ο πλήρης πηγαίος κώδικας της εφαρμογής, συμπεριλαμβανομένων των ρυθμίσεων του Docker, φιλοξενείται στην πλατφόρμα GitHub και είναι προσβάσιμος στη

διεύθυνση: https://github.com/iosifidis/msc-pims

4.2 Εκκίνηση έργου (Project Initialization)

Η διαδικασία εκκίνησης του έργου πραγματοποιήθηκε με τη βοήθεια εργαλείων αυτοματοποίησης εντός του VS Code. Μέσω της Παλέτας Εντολών (Command Palette) και της διασύνδεσης με το **Spring Initializr**, δημιουργήθηκε ο σκελετός της εφαρμογής βασισμένος στην έκδοση Java 21 και αρχικά στην έκδοση Spring Boot 3.3.x.

Σε αυτό το πρώιμο στάδιο, η Τεχνητή Νοημοσύνη αξιοποιήθηκε για τη διαμόρφωση της σύνδεσης με τη βάση δεδομένων. Δόθηκε στο Gemini το ακόλουθο τεχνικό Prompt:

> *"Generate the application.properties configuration for a Spring Boot application connecting to a PostgreSQL database running on localhost:5432, database name 'vet_db', with username 'user' and password 'pass'."*

Το μοντέλο παρήγαγε με ακρίβεια τις απαραίτητες ρυθμίσεις (spring.datasource.url, username, password, driver-class-name), επιβεβαιώνοντας την ικανότητά του να διαχειρίζεται τυπικές εργασίες διαμόρφωσης (Infrastructure as Code tasks). Η επιτυχής εκκίνηση της εφαρμογής και η σύνδεση με τον Docker container της βάσης αποτέλεσαν την πρώτη επιβεβαίωση της λειτουργικότητας του περιβάλλοντος.

4.3 Υλοποίηση Backend: Προκλήσεις και συνεργασία με την ΤΝ

Η διαδικασία ανάπτυξης του Backend δεν περιορίστηκε σε μια παθητική αποδοχή του παραγόμενου κώδικα από το μοντέλο Gemini 3 Pro. Αντιθέτως, εξελίχθηκε σε μια δυναμική διαδικασία «δοκιμής και σφάλματος» (trial and error), όπου ο ανθρώπινος παράγοντας είχε τον ρόλο του τελικού κριτή και διορθωτή. Μέσα από την ανάλυση των αρχείων καταγραφής (development logs), αναδείχθηκαν τρεις χαρακτηριστικές περιπτώσεις (case studies) που αποδεικνύουν την ανάγκη για κριτική σκέψη και βαθιά τεχνική γνώση κατά τη χρήση Μεγάλων Γλωσσικών Μοντέλων (LLMs) στη μηχανική λογισμικού.

4.3.1 Case Study 1: Διαχείριση συσχετίσεων Entity και JSON Serialization (The Infinite Recursion Problem)

Κατά την υλοποίηση της αμφίδρομης συσχέτισης «Ένα-προς-Πολλά» (One-to-Many) μεταξύ των οντοτήτων Client (Κηδεμόνας) και Patient (Ασθενής), το σύστημα οδηγήθηκε σε κατάρρευση λόγω σφάλματος StackOverflowError.

Το πρόβλημα προέκυψε επειδή η ΤΝ, ακολουθώντας τις οδηγίες για τη δημιουργία

των JPA Entities, δημιούργησε σωστά τη σχέση στη βάση δεδομένων, αλλά αγνόησε τις επιπτώσεις κατά τη διαδικασία σειριοποίησης (serialization) των αντικειμένων σε μορφή JSON. Συγκεκριμένα, κατά την κλήση ενός API endpoint για την ανάκτηση ενός Client, ο Jackson serializer προσπαθούσε να μετατρέψει τον Client σε JSON, ο οποίος περιείχε μια λίστα patients. Κάθε Patient στη λίστα, ωστόσο, περιείχε μια αναφορά πίσω στον Client, οδηγώντας σε έναν ατέρμονα βρόχο αναφορών.

Η λύση δόθηκε μέσω ανθρώπινης παρέμβασης, με την εισαγωγή των ειδικών annotations @JsonManagedReference και @JsonBackReference της βιβλιοθήκης Jackson, τα οποία καθοδηγούν τον serializer να αγνοεί την «πίσω» αναφορά κατά τη μετατροπή, σπάζοντας τον κύκλο.

Snippet Κώδικα - Το Λάθος (Αρχική Πρόταση AI):

```
// Entity: Client.java
@OneToMany(mappedBy = "owner")
private List<Patient> pets; // Το AI δεν έβαλε προστασία εδώ

// Entity: Patient.java
@ManyToOne
@JoinColumn(name = "owner_id")
private Client owner; // Ούτε εδώ
```

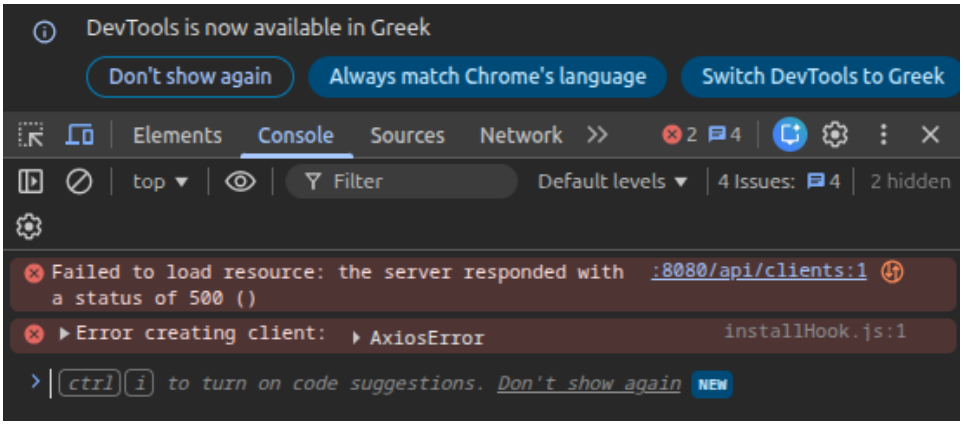
Snippet Κώδικα - Η Διόρθωση (Ανθρώπινη Παρέμβαση):

```
// Entity: Client.java
@OneToMany(mappedBy = "owner", cascade = CascadeType.ALL)
@JsonManagedReference // Λύση: Ορίζει ότι αυτή η πλευρά σειριοποιείται
private List<Patient> pets;

// Entity: Patient.java
@ManyToOne
@JoinColumn(name = "owner_id")
@JsonBackReference // Λύση: Ορίζει ότι αυτή η πλευρά αγνοείται στο JSON
private Client owner;
```

Create a ClientController in package com.pims.backend.controller. Annotate it with @RestController and @RequestMapping("/api/clients"). Inject ClientRepository. Create a GET endpoint getAllClients that returns List<Client>. Allow CORS from http://localhost:3000 (for our React frontend later) using @CrossOrigin.

Πίνακας 4.1: Prompt για τη δημιουργία του Client Controller



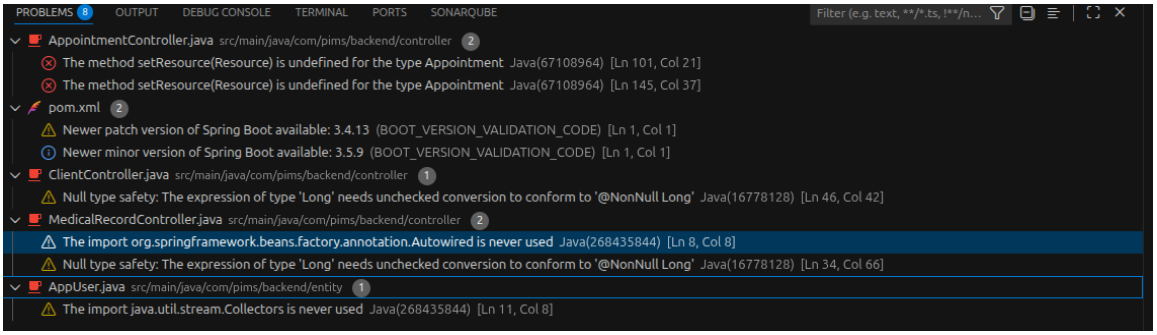
Εικόνα 4.1: Σφάλμα διακομιστή (500 Internal Server Error) λόγω άπειρης αναδρομής κατά τη σειριοποίηση JSON

4.3.2 Case Study 2: Η Στρατηγική Απόφαση αφαίρεσης του Lombok (Refactoring)

Ημ/νια	Τύπος Σφάλματος	Περιγραφή / Μήνυμα Συστήματος	Ενέργεια Επίλυσης
14/12	Logic Error	StackOverflowError (Infinite Recursion)	Χρήση @JsonManagedReference & @JsonBackReference
26/12	Compilation Error	Cannot find symbol: method builder()	Αφαίρεση βιβλιοθήκης Lombok & Refactoring σε POJOs
27/12	Hallucination	PropertyReferenceException: No property 'client' found	Χειροκίνητη διόρθωση JPQL query (owner αντί client)

Πίνακας 4.2: Καταγραφή κρίσιμων σφαλμάτων κατά την ανάπτυξη του Backend

Μια από τις πιο σημαντικές αρχιτεκτονικές αποφάσεις ελήφθη το διήμερο 26-27 Δεκεμβρίου, αφορώντας τη χρήση της βιβλιοθήκης **Project Lombok**. Στην αρχική φάση του έργου, το Gemini πρότεινε και εφάρμοσε αυτόματα τη χρήση Lombok annotations (@Data, @Builder, @NoArgsConstructor) για τη μείωση του boilerplate κώδικα, μια πρακτική που θεωρείται standard στη βιομηχανία.



Εικόνα 4.2: Σφάλματα μεταγλώττισης (Compilation Errors) λόγω αποτυχίας του Lombok annotation processor.

Act as a Senior Java Developer. The Lombok library is failing in the user's environment. We must replace Lombok annotations with standard Java boilerplate. *Task: Rewrite the following Entity classes using Standard Java (No @Data, No @Builder, No @AllArgsConstructor). Requirements:* *Add explicit private fields.* *Add an explicit Empty Constructor (public ClassName() {}).* *Add an explicit Full Constructor.* *Add standard Getters and Setters for all fields.* *Files to Rewrite:* *src/main/java/com/pims/backend/entity/AppUser.java (Fields: id, username, passwordHash, fullName, email, role, isActive).* *src/main/java/com/pims/backend/entity/Role.java (Fields: id, name).* *src/main/java/com/pims/backend/entity/Client.java (Fields: id, firstName, lastName, email, phone, address).* *src/main/java/com/pims/backend/entity/Patient.java (Fields: id, name, species (Enum), breed, sex (Enum), owner (Client)).* *src/main/java/com/pims/backend/entity/Appointment.java (Fields: id, startTime, endTime, reason, status (Enum), client, patient, vet).* *src/main/java/com/pims/backend/entity/MedicalRecord.java (Fields: id, diagnosis, treatment, weight, temperature, symptoms, notes, appointment, patient).* *Output: Provide the FULL CODE for all 6 files.*

Πίνακας 4.3: Prompt για την αντικατάσταση του Lombok με Standard Java.

Ωστόσο, η ενσωμάτωση αυτή δημιούργησε ένα φαινόμενο «Dependency Hell». Όπως καταγράφεται στα logs, το περιβάλλον ανάπτυξης (VS Code) αδυνατούσε να συγχρονιστεί σωστά με τον annotation processor, οδηγώντας σε ψευδή θετικά σφάλματα (false positives) και, κυρίως, σε πραγματικά compilation errors όπου οι μέθοδοι builder() δεν αναγνωρίζονταν από τους Controllers.

Αντί να αναλωθεί χρόνος στην παραμετροποίηση του IDE, ο προγραμματιστής έλαβε τη στρατηγική απόφαση για **απλοποίηση της αρχιτεκτονικής**. Ζητήθηκε ρητά από το AI να προχωρήσει σε καθολικό Refactoring, αφαιρώντας το Lombok και ξαναγράφοντας όλα τα Entities σε **Plain Old Java Objects (POJOs)**.

Το σχετικό prompt που δόθηκε στο AI ήταν καθοριστικό:

> *"Fix this Entity class. It is causing compilation errors in the Controller. Task: Rewrite Appointment.java using Standard Java (No Lombok). CRITICAL: Generate public void setResource... and Getters and Setters for ALL other fields.".*

Αυτή η κίνηση απέδειξε ότι ο άνθρωπος-προγραμματιστής πρέπει να είναι σε θέση να αξιολογεί το κόστος συντήρησης των εργαλείων που προτείνει η ΤΝ και να επιβάλλει λύσεις που εξασφαλίζουν τη σταθερότητα του έργου, ακόμη και αν αυτό σημαίνει περισσότερο όγκο κώδικα.

4.3.3 Case Study 3: Αντιμετώπιση παραισθήσεων της ΤΝ (AI Hallucinations)

Κατά την ανάπτυξη των Repositories, εντοπίστηκε ένα χαρακτηριστικό παράδειγμα «παραισθήσεων» (Hallucinations) του μοντέλου. Η ΤΝ προσπάθησε να δημιουργήσει ένα Derived Query στο PatientRepository για την εύρεση ασθενών βάσει ιδιοκτήτη.

Ενώ στο Entity Patient το πεδίο που αναφέρεται στον ιδιοκτήτη είχε ονομαστεί **owner** (βάσει της ανθρώπινης σχεδίασης), η ΤΝ «υπέθεσε» αυθαίρετα ότι το πεδίο ονομαζόταν client, ακολουθώντας πιθανώς πιο συνηθισμένα μοτίβα που είχε δει στα δεδομένα εκπαίδευσής του.

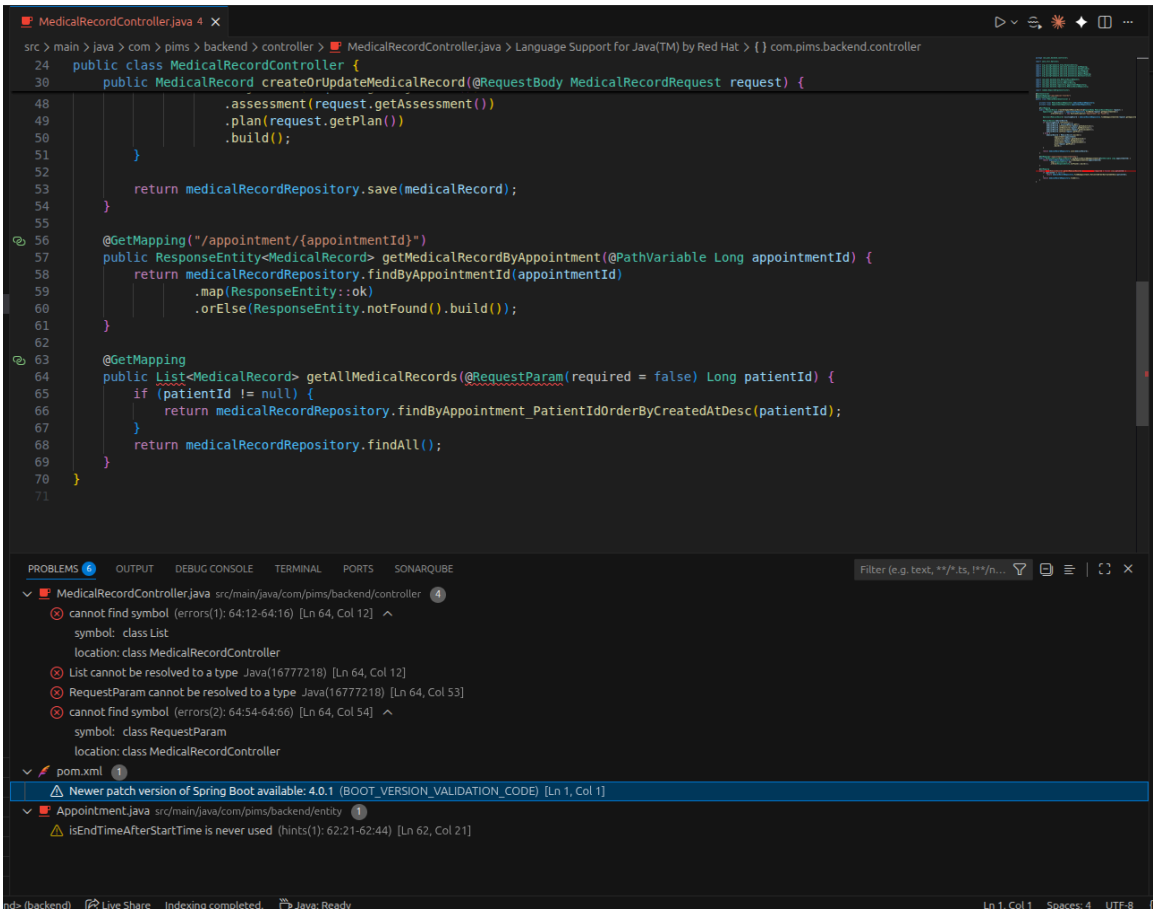
Το Σφάλμα: Το AI παρήγαγε τη μέθοδο:
List<Patient> findByClientId(Long clientId);

Αυτό προκάλεσε σφάλμα εκκίνησης του Spring Context: PropertyReferenceException: No property 'client' found for type 'Patient'.

Η ανάλυση & διόρθωση: Το μοντέλο απέτυχε να διαβάσει με ακρίβεια το context του αρχείου Patient.java που του είχε δοθεί και βασίστηκε σε γενικευμένη γνώση. Η διόρθωση απαιτούσε την παρέμβαση του προγραμματιστή, ο οποίος εντόπισε την ασυμφωνία ονοματολογίας και ζήτησε τη διόρθωση του query ώστε να αντικατοπτρίζει το πραγματικό σχήμα δεδομένων:
List<Patient> findByOwnerId(Long ownerId);

Το περιστατικό αυτό υπογραμμίζει τον κίνδυνο της τυφλής εμπιστοσύνης στον κώδικα που παράγεται από τα Μεγάλα Γλωσσικά Μοντέλα (LLMs), ειδικά όταν πρόκειται για τη διασύνδεση διαφορετικών εξαρτημάτων του συστήματος (στην

προκειμένη περίπτωση, Entity και Repository), όπου η ακρίβεια στην ονοματολογία είναι κρίσιμη.



Εικόνα 4.3: Παράδειγμα ελλιπούς κώδικα από την TN (απουσία απαραίτητων imports)

Act as a Senior Spring Boot Developer. The application failed to start with PropertyReferenceException: No property 'clientId' found for type 'Patient'.

The Issue: The derived query method findByPatientClientIdOrderByCreatedAtDesc is failing because Spring Data JPA is confused about the property path.

Task: Fix src/main/java/com/pims/backend/repository/MedicalRecordRepository.java.

The Solution: Instead of a complex derived method name, use a simple @Query annotation (JPQL) to be explicit and safe.

Code to Generate:

Java

```
@Repository
public interface MedicalRecordRepository extends JpaRepository<MedicalRecord, Long> {
// Explicit JPQL is safer here
@Query("SELECT m FROM MedicalRecord m WHERE m.patient.client.id = :clientId ORDER BY m.createdAt DESC")
List<MedicalRecord> findByClientId(@Param("clientId") Long clientId);
// Keep existing methods if any, or replace the broken one.
```

```
Optional<MedicalRecord> findByAppointmentId(Long appointmentId);
List<MedicalRecord> findByPatientId(Long patientId);
}


**Also Update Controller:** Update `MedicalRecordController.java` to call this new method
`findByClientId` instead of the long broken one.
Output: Provide the FULL CODE for both MedicalRecordRepository.java and
MedicalRecordController.java.
```

Πίνακας 4.4: Prompt διόρθωσης λανθασμένου Query (Hallucination)

4.4 Υλοποίηση ασφάλειας και διαχείριση πρόσβασης (Security & Authorization Challenges)

Η ενσωμάτωση του **Spring Security** αποτέλεσε μία από τις πιο απαιτητικές φάσεις της υλοποίησης. Σε αντίθεση με την παραγωγή απλού κώδικα CRUD, η ασφάλεια απαιτεί λεπτομερή παραμετροποίηση που εξαρτάται από την αρχιτεκτονική του συστήματος. Η διαδικασία δεν ήταν γραμμική, αλλά ακολούθησε έναν επίπονο επαναληπτικό κύκλο εντοπισμού σφαλμάτων και ρύθμισης, με την ΤΝ να παρέχει τον βασικό κορμό και τον προγραμματιστή να επιλύει τα προβλήματα ολοκλήρωσης.

4.4.1 Η μάχη με τα CORS και CSRF: Η επικοινωνία Frontend-Backend

Το πρώτο σημαντικό εμπόδιο εμφανίστηκε αμέσως μόλις επιχειρήθηκε η σύνδεση του Frontend (localhost:5173) με το Backend (localhost:8080). Κάθε προσπάθεια αποστολής αιτήματος (Request) κατέληγε σε σφάλμα δικτύου ή απόρριψη από τον διακομιστή.

Το Gemini, ακολουθώντας τις προεπιλεγμένες ρυθμίσεις ασφαλείας του Spring Security 6.x, παρήγαγε αρχικά μια διαμόρφωση που απέκλειε κάθε εξωτερική πηγή. Το πρόβλημα εντοπίστηκε σε δύο επίπεδα:

- **CORS (Cross-Origin Resource Sharing):** Ο browser μπλόκαρε τα αιτήματα επειδή προέρχονταν από διαφορετική προέλευση (port 5173) από αυτή του διακομιστή.
- **CSRF (Cross-Site Request Forgery):** Η ΤΝ είχε ενεργοποιημένη την προστασία CSRF, η οποία όμως είναι σχεδιασμένη για session-based εφαρμογές και όχι για stateless APIs που βασίζονται σε JWT, οδηγώντας σε απόρριψη των POST αιτημάτων.

Η λύση δόθηκε μέσω ρητής, τεχνικής καθοδήγησης προς την ΤΝ. Ο προγραμματιστής ζήτησε συγκεκριμένα: *"Disable CSRF protection as we are using JWT, and configure CORS to allow requests specifically from http://localhost:5173"*. Η προσαρμογή του SecurityFilterChain βάσει αυτών των οδηγιών επέτρεψε την πρώτη επιτυχή "συνομιλία" μεταξύ των δύο υποσυστημάτων.

4.4.2 Το πρόβλημα των "403 Forbidden" και τα Request Matchers

Ακόμη και μετά την επίλυση των ζητημάτων δικτύωσης, το σύστημα συνέχισε να επιστρέφει σφάλματα **403 Forbidden** σε αιτήματα που φαινομενικά διέθεταν έγκυρα tokens. Η ανάλυση των logs έδειξε ότι το πρόβλημα δεν ήταν στην αυθεντικοποίηση (Authentication), αλλά στην εξουσιοδότηση (Authorization).

Η αρχική διαμόρφωση της ΤΝ ήταν υπερβολικά αυστηρή ή ασαφής ως προς τα endpoints. Όπως καταγράφεται στα logs, χρειάστηκε να δοθεί η εντολή: *"Fix the 403 Forbidden error on POST /api/clients. Ensure that authenticated users... can access /api/clients/**"*.

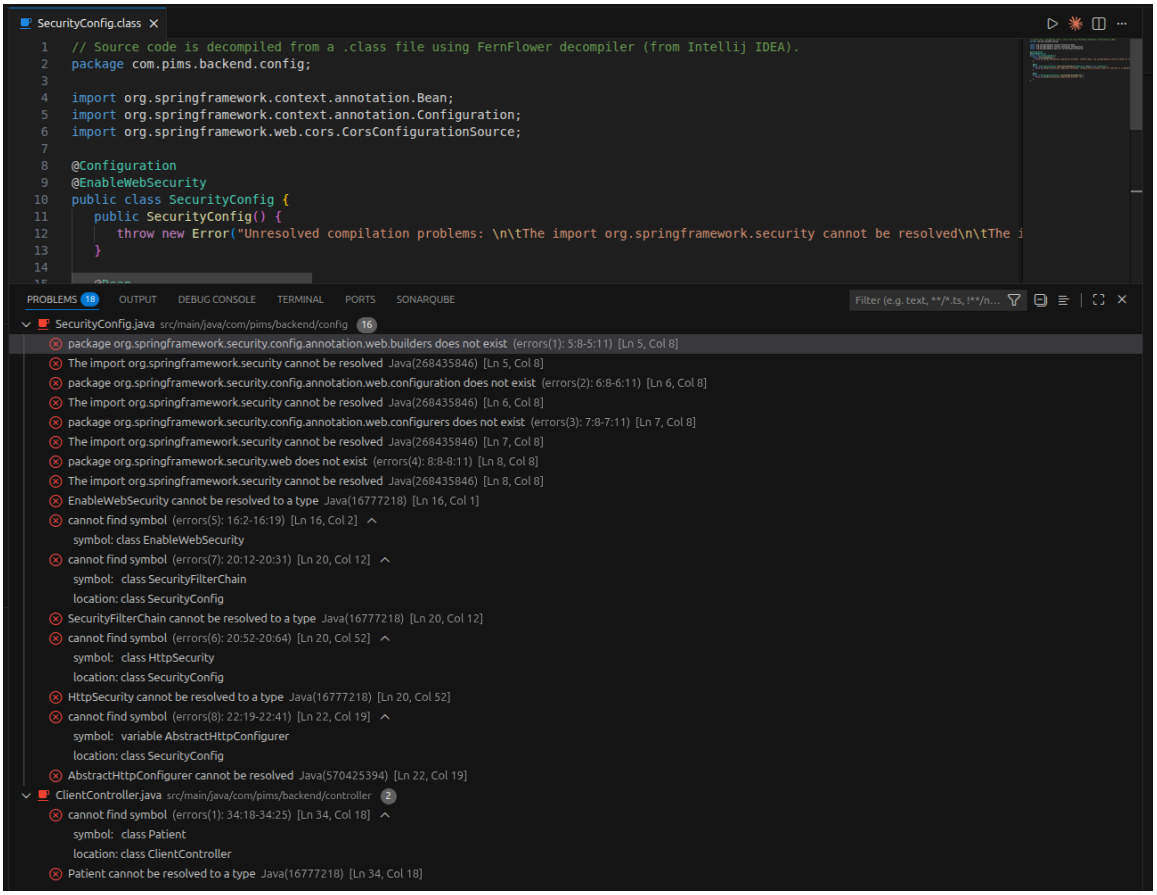
Μέσα από διαδοχικές δοκιμές, χαρτογραφήθηκαν σωστά τα requestMatchers στο αρχείο SecurityConfig.java, διαχωρίζοντας τα δημόσια endpoints (π.χ. /api/auth/**) από τα προστατευμένα (π.χ. /api/dashboard/**), διασφαλίζοντας ότι το Spring Context αναγνωρίζει σωστά τη ροή πρόσβασης.

4.4.3 Θωράκιση του JWT Filter: Από το Crash στη διαχείριση εξαιρέσεων

Η κρισιμότερη παρέμβαση για τη σταθερότητα του συστήματος πραγματοποιήθηκε στο επίπεδο του **JwtAuthenticationFilter**. Κατά τη διάρκεια δοκιμών με λανθασμένα ή παραποιημένα tokens, παρατηρήθηκε ότι το Backend κατέρρεε επιστρέφοντας **500 Internal Server Error** αντί για το αναμενόμενο 403/401. Η αιτία ήταν η ρίψη της εξαίρεσης MalformedJwtException από τη βιβλιοθήκη jjwt, η οποία διέκοπτε βίαια την εκτέλεση του φίλτρου.

Η ΤΝ είχε γράψει τον κώδικα για το "Happy Path" (έγκυρο token), αλλά είχε παραμελήσει τη διαχείριση σφαλμάτων. Η ανθρώπινη παρέμβαση ήταν καθοριστική: ζητήθηκε η προσθήκη try-catch blocks γύρω από τη διαδικασία ανάλυσης του token (parsing).

Η οδηγία στο log ήταν χαρακτηριστική για την τεχνική ωριμότητα της παρέμβασης: *"Just call filterChain.doFilter(request, response) in the finally block or after the catch. This ensures the request reaches the SecurityConfig, which will send a proper 403 instead of a 500 crash"*. Με αυτή τη διόρθωση, το σύστημα απέκτησε την ανθεκτικότητα που απαιτείται σε ένα παραγωγικό περιβάλλον, διαχειριζόμενο κομψά τις προσπάθειες μη εξουσιοδοτημένης πρόσβασης.



Εικόνα 4.4: Προβλήματα συμβατότητας εκδόσεων Spring Security κατά τη μετάβαση σε Spring Boot 3

Act as a Senior Spring Boot Developer. I need the final, error-free version of SecurityConfig.java.

Context:

I am using Spring Boot 3 and Spring Security 6.

My user repository is named AppUserRepository (not UserRepository).

I have a custom filter named JwtAuthenticationFilter.

The Frontend runs on http://localhost:5173.

Requirements for the Code:

Imports: Ensure java.util.List, DaoAuthenticationProvider, and all Security classes are imported correctly.

Beans:

Define UserDetailsService bean that uses appUserRepository.findByUsername().

Define AuthenticationProvider (use DaoAuthenticationProvider).

Define AuthenticationManager.

Define PasswordEncoder (BCryptPasswordEncoder).

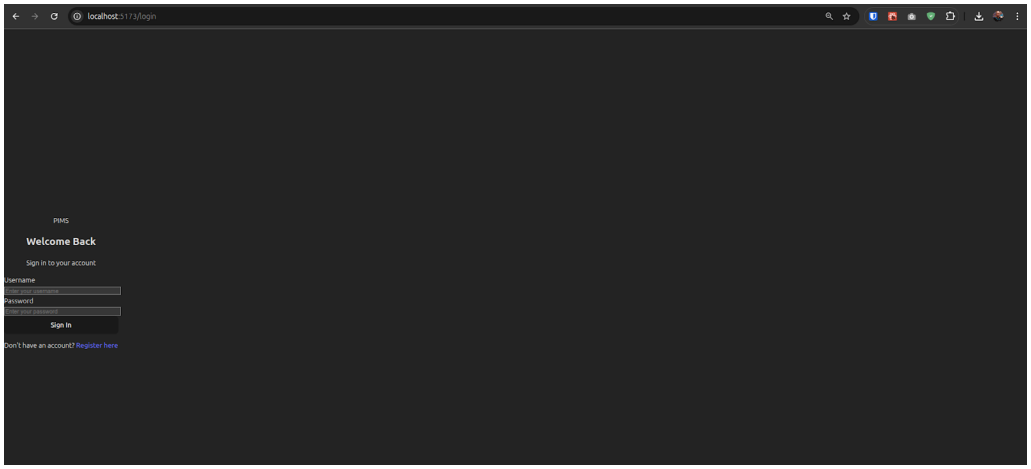
SecurityFilterChain: Disable CSRF (using AbstractHttpConfigurer::disable). Configure CORS to allow http://localhost:5173 (GET, POST, PUT, DELETE, OPTIONS) and allow credentials. Set session policy to STATELESS. Allow access to /api/auth/**. Add jwtAuthFilter before UsernamePasswordAuthenticationFilter. Please provide the full SecurityConfig.java code.

Πίνακας 4.5: Prompt για την τελική παραμετροποίηση του Spring Security 6.

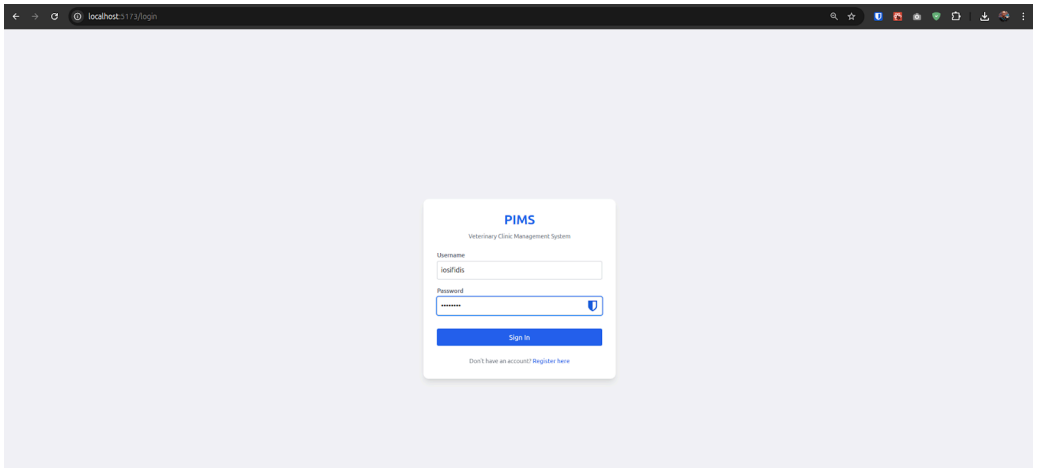
4.5 Ανάπτυξη διεπαφής χρήστη με λειτουργικά Prompts (Frontend Development)

Η μετάβαση από την ανάπτυξη του Backend στην υλοποίηση του Frontend σηματοδότησε μια ουσιώδη αλλαγή στη στρατηγική αλληλεπίδρασης με το μοντέλο Τεχνητής Νοημοσύνης. Ενώ στο Backend τα prompts ήταν αυστηρά δομικά και τεχνικά (π.χ. «Δημιούργησε την κλάση Entity με συγκεκριμένα πεδία»), στο Frontend υιοθετήθηκε η μεθοδολογία των **Λειτουργικών Prompts (Functional Prompting)**.

Σε αυτή τη φάση, η ΤΝ κλήθηκε να λειτουργήσει ταυτόχρονα ως **Προγραμματιστής (Coder)** και ως **Σχεδιαστής διεπαφής (UI Designer)**. Τα prompts βασίστηκαν αυτούσια στα User Stories της ανάλυσης, περιγράφοντας την επιθυμητή συμπεριφορά και αισθητική, αφήνοντας τις λεπτομέρειες υλοποίησης στην κρίση του μοντέλου.



Εικόνα 4.5: Η αρχική, απλοϊκή μορφή της σελίδας εισόδου (Basic Login)

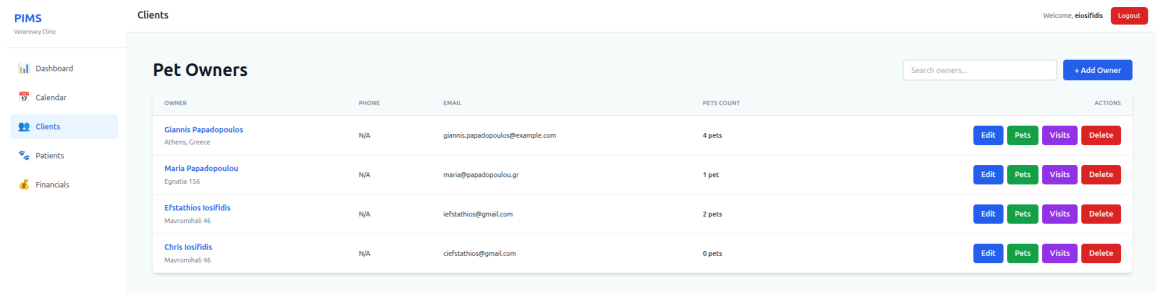


Εικόνα 4.6: Η τελική, “επαγγελματική” μορφή της σελίδας εισόδου

Χαρακτηριστικά παραδείγματα αυτής της προσέγγισης, όπως καταγράφονται στο ημερολόγιο ανάπτυξης, περιλαμβάνουν:

- **Λειτουργική περιγραφή:** *"Create a Dashboard with 3 summary cards for Total Patients, Appointments"*. Η ΤΝ αντιλήφθηκε την ανάγκη για συνοπτική παρουσίαση πληροφορίας και παρήγαγε αυτόματα τη δομή Grid με τα αντίστοιχα React components.
- **Οπτική καθοδήγηση:** *"Redesign Login"*. Εδώ, η ΤΝ λειτούργησε ως Designer, μετατρέποντας μια απλή φόρμα HTML σε μια σύγχρονη σελίδα με σκιές, στρογγυλεμένες γωνίες και μοντέρνα τυπογραφία, χρησιμοποιώντας βιβλιοθήκες CSS (όπως Tailwind) χωρίς να της ζητηθεί ρητά ο κώδικας του στυλ.

Επιπλέον, η ΤΝ πρότεινε και ενσωμάτωσε εξειδικευμένες βιβλιοθήκες για την επίλυση σύνθετων UI προβλημάτων, όπως το react-big-calendar για τη διαχείριση του χρονοδιαγράμματος και το date-fns για τον χειρισμό των ημερομηνιών, επιταχύνοντας δραματικά τον χρόνο ανάπτυξης.



Εικόνα 4.7: Η οθόνη διαχείρισης κηδεμόνων (Client Management) με ενσωματωμένα εργαλεία διαχείρισης (Edit, Pets, Visits, Delete)

4.6 Επίλυση προβλημάτων ενοποίησης (Integration Debugging)

Η φάση της ενοποίησης του React Frontend (localhost:5173) με το Spring Boot Backend (localhost:8080) ανέδειξε σημαντικές τεχνικές προκλήσεις, χαρακτηριστικές των κατανεμημένων αρχιτεκτονικών.

Το κυριότερο εμπόδιο ήταν η ασφάλεια επικοινωνίας. Αρχικά, οι φυλλομετρητές απέκλειαν τα αιτήματα λόγω πολιτικής **CORS (Cross-Origin Resource Sharing)**. Ακόμη και μετά τη ρύθμιση του CORS, το σύστημα επέστρεφε σφάλματα **403 Forbidden**, καθώς το Backend αδυνατούσε να επαληθεύσει την ταυτότητα του χρήστη.

```
Access to XMLHttpRequest at 'http://localhost:8080/api/clients'   (index):1
from origin 'http://localhost:5173' has been blocked by CORS policy: No
'Access-Control-Allow-Origin' header is present on the requested resource.
▶Error fetching data:                                              installHook.js:1
  ▶AxiosError {message: 'Network Error', name: 'AxiosError', code: 'ERR_N
   ETWORK', config: {…}, request: XMLHttpRequest, …}
▶GET http://localhost:8080/api/clients net::ERR_FAILED 403   App.jsx:10
(Forbidden)
Access to XMLHttpRequest at 'http://localhost:8080/api/clients'   (index):1
from origin 'http://localhost:5173' has been blocked by CORS policy: No
'Access-Control-Allow-Origin' header is present on the requested resource.
▶Error fetching data:                                              installHook.js:1
  ▶AxiosError {message: 'Network Error', name: 'AxiosError', code: 'ERR_N
   ETWORK', config: {…}, request: XMLHttpRequest, …}
▶GET http://localhost:8080/api/clients net::ERR_FAILED 403   App.jsx:10
(Forbidden)
```

Εικόνα 4.8: Σφάλματα CORS στον φυλλομετρητή που εμπόδιζαν την επικοινωνία Frontend-Backend.

Η ανάλυση των σφαλμάτων έδειξε ότι το Frontend έστελνε λανθασμένα headers (π.χ. Bearer null ή Bearer undefined), οδηγώντας το Backend σε εξαιρέσεις (MalformedJwtException). Η λύση δόθηκε μέσω της υλοποίησης του προτύπου **Axios Interceptor**, με την καθοδήγηση της ΤΝ.

Συγκεκριμένα, ζητήθηκε από την ΤΝ να δημιουργήσει έναν κεντρικό μηχανισμό αναχαίτισης αιτημάτων (axiosConfig.js), ο οποίος:

1. Ελέγχει την ύπαρξη του JWT Token στο localStorage πριν από κάθε αίτημα.
2. Φιλτράρει άκυρες τιμές (όπως τα strings "null" ή "undefined").
3. Επισυνάπτει αυτόματα το σωστό Authorization header.

```
react-dom_client.js?v=39209900:20103
Download the React DevTools for a better development experience:
https://react.dev/link/react-devtools
Failed to load resource: the server responded with a status of 403 ()   :8080/api/auth/login:1
▸Login failed: ▸AxiosError   AuthContext.jsx:45
Sending registration payload: ▸Object   RegisterPage.jsx:42
Failed to load resource: the server responded with a status of 403 ()   :8080/api/auth/register:1
▾Registration failed: ▸AxiosError   AuthContext.jsx:56
  register @ AuthContext.jsx:56
▾Registration error: ▸AxiosError   RegisterPage.jsx:46
  handleSubmit @ RegisterPage.jsx:46
▾Error response data:   RegisterPage.jsx:47
  handleSubmit @ RegisterPage.jsx:47
Failed to load resource: the server responded with a status of 403 ()   :8080/api/auth/login:1
▾Login failed: ▸AxiosError   AuthContext.jsx:45
  login @ AuthContext.jsx:45
▸POST http://localhost:8080/api/auth/login 404 (Not Found)   AuthContext.jsx:32
▸Login failed:   AuthContext.jsx:45
  ▸AxiosError {message: 'Request failed with status code 404', name: 'AxiosError', code: 'ERR_BAD_REQUEST', config: {…}, request: XMLHttpRequest, …}
>
```

Εικόνα 4.9: Αποτυχία εισόδου λόγω προβλημάτων αυθεντικοποίησης (Authentication Failure) και εσφαλμένης διαχείρισης του JWT Token.

Αυτή η παρέμβαση μετέτρεψε το AI από απλό παραγωγό κώδικα σε εργαλείο επίλυσης προβλημάτων υποδομής (Infrastructure debugger), γεφυρώνοντας το χάσμα μεταξύ των δύο τεχνολογικών στοιβών.

4.7 Η ΤΝ ως Αρχιτέκτονας Λογισμικού. Η φάση του Antigravity

Στο τελικό στάδιο της ανάπτυξης, εφαρμόστηκε πρόσβαση του συνόλου του πηγαίου κώδικα με χρήση του **"Antigravity"** (νέο IDE της Google). Εκμεταλλευόμενοι το μεγάλο παράθυρο περιβάλλοντος (large context window) του Gemini 3 Pro,

τροφοδοτήσαμε το μοντέλο με το **σύνολο της βάσης κώδικα (codebase)**, συμπεριλαμβανομένων τόσο των Backend όσο και των Frontend αρχείων.

Σε αυτό το πλαίσιο, ζητήθηκε από την ΤΝ να εγκαταλείψει τον ρόλο του προγραμματιστή και να αναλάβει τον ρόλο του **Αρχιτέκτονα Λογισμικού (Software Architect)**. Ο στόχος ήταν η εκτέλεση ανάλυσης κενών και η δημιουργία σχεδίων για την ολοκλήρωση του έργου.

Η ΤΝ ανταποκρίθηκε παράγοντας δύο κρίσιμα αρχιτεκτονικά τεκμήρια, τα οποία αποδεικνύουν βαθιά κατανόηση της επιχειρησιακής λογικής:

Α. Σχέδιο υλοποίησης (Implementation Plan):
Η ΤΝ εντόπισε ελλείψεις σε σχέση με τα αρχικά Epics και πρότεινε συγκεκριμένες τροποποιήσεις αρχείων. Όπως φαίνεται στο αρχείο implementation_plan, η ΤΝ πρότεινε:

- **Client Management:** Την υλοποίηση "Cascade Delete" για τον καθαρισμό δεδομένων κατά τη διαγραφή πελάτη (Client.java).
- **Patient Management:** Την προσθήκη λειτουργίας μεταβίβασης ιδιοκτησίας ζώου (Transfer Ownership) και την υποστήριξη κατά προσέγγιση ημερομηνιών γέννησης (Year-Only Birth Date).
- **Clinical Operations:** Την προσθήκη λειτουργίας Follow-up ραντεβού μέσα από το Medical Modal.

Proposed Changes

Backend

[Client Management]

- Goal: Enable Cascade Deletion for Clients (US 5.3).

- Files:

- [MODIFY] [Client.java]: Add `@OneToMany` relationship for appointments with `CascadeType.ALL`.

[Patient Management]

- Goal: Enable Patient Ownership Transfer (US 6.2).

- Files:

- [MODIFY] [PatientController.java]: Add `PUT /api/patients/{id}/owner/{ownerId}` endpoint.

- [MODIFY] [PatientService.java]: Add method signature.

- [MODIFY] [PatientServiceImpl.java]: Implement transfer logic.

[Appointment Management]

- Goal: Enforce Task Locking Logic (US 3.4).

- Files:

- [MODIFY] [AppointmentServiceImpl.java]: Update `updateAppointment` to check for locking conditions. Allow unlock only if rescheduling to future.

Frontend

[Client Management]

- Goal: Add Stray Animal Caretaker tag (US 5.1).

- Files:

- [MODIFY] [ClientForm.jsx]: Add checkbox for `isStrayCaretaker`.

[Pet Management]

- Goal: Support Year-Only Birth Date (US 6.1).

- Files:

- [MODIFY] [ClientsPage.jsx]: Update `PetManagerModal` to include "Approximate Date / Year only" toggle and logic.

[Appointment Management]

- Goal: Improve Appointment Booking UX (US 4.1).

- Files:

- [MODIFY] [AppointmentForm.jsx]: Refactor to use search-based selection for Client/Patient and dropdowns for Vet/Resource.

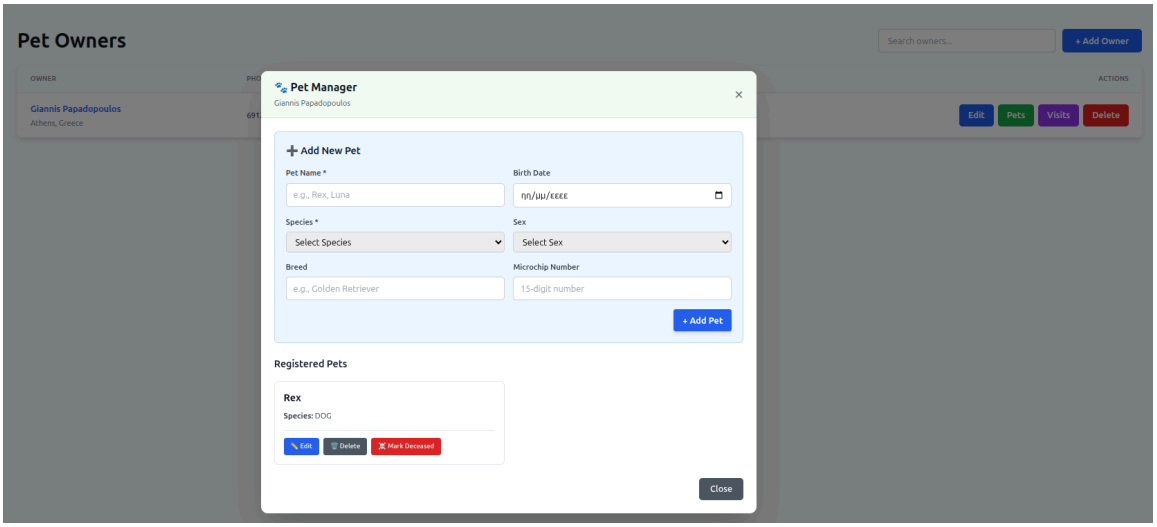
[Clinical Operations]

- Goal: Add Follow-up and Quick Booking (US 3.6).

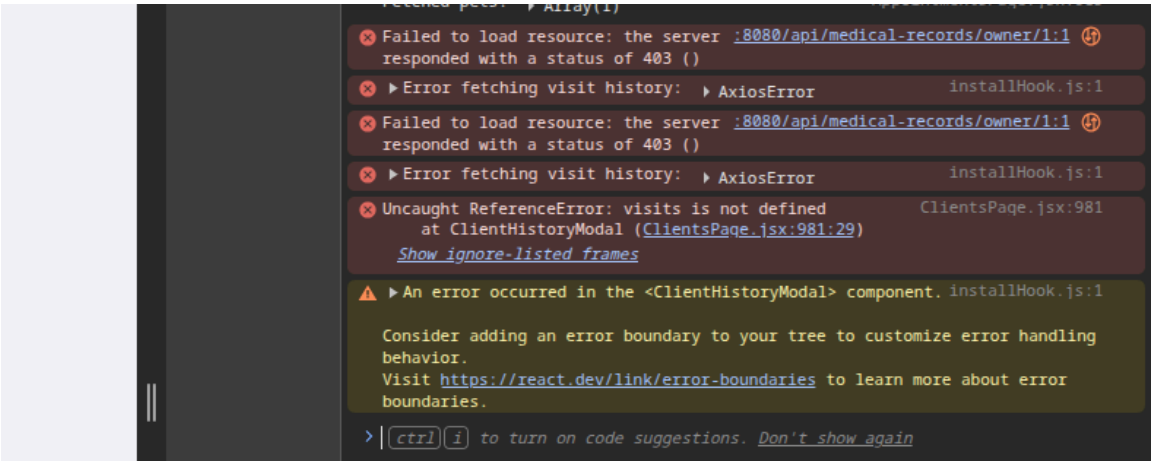
- Files:

- [MODIFY] [MedicalRecordModal.jsx]: Add "Follow-up Date" field and "Book Next Appointment" button.

Πίνακας 4.6: Απόσπασμα από το σχέδιο υλοποίησης (Implementation Plan) που παρήγαγε η TN.



Εικόνα 4.10: Η φόρμα διαχείρισης ασθενών που δημιουργήθηκε για την επίλυση προβλημάτων συσχέτισης



Εικόνα 4.11: Σφάλμα λογικής στο Frontend (ReferenceError) που εντοπίστηκε κατά την ανάπτυξη

Β. Οδηγός επαλήθευσης (Walkthrough Report):

Η ΤΝ δημιούργησε ένα σενάριο ελέγχου ποιότητας (walkthrough), καθορίζοντας τα βήματα για τη χειροκίνητη επαλήθευση των νέων λειτουργιών. Για παράδειγμα, για το "Appointment Locking", καθόρισε τη διαδικασία δοκιμής επεξεργασίας ενός παρελθοντικού ραντεβού για να επιβεβαιώσει ότι το σύστημα το κλειδώνει σωστά.

Η φάση αυτή καταδεικνύει ένα ανώτερο επίπεδο συνεργασίας ανθρώπου-μηχανής. Η ΤΝ δεν παρήγαγε απλώς αποσπασματικό κώδικα, αλλά κατανόησε τις αλληλεξαρτήσεις ολόκληρου του έργου, πρότεινε αρχιτεκτονικές βελτιώσεις και συνέταξε τεχνική τεκμηρίωση, λειτουργώντας ως ένας πραγματικός συνεργάτης υψηλού επιπέδου.

PIMS Feature Implementation Walkthrough

This document outlines the changes made to the Practice Information Management System (PIMS) and provides instructions on how to verify them.

1. Client Management Updates

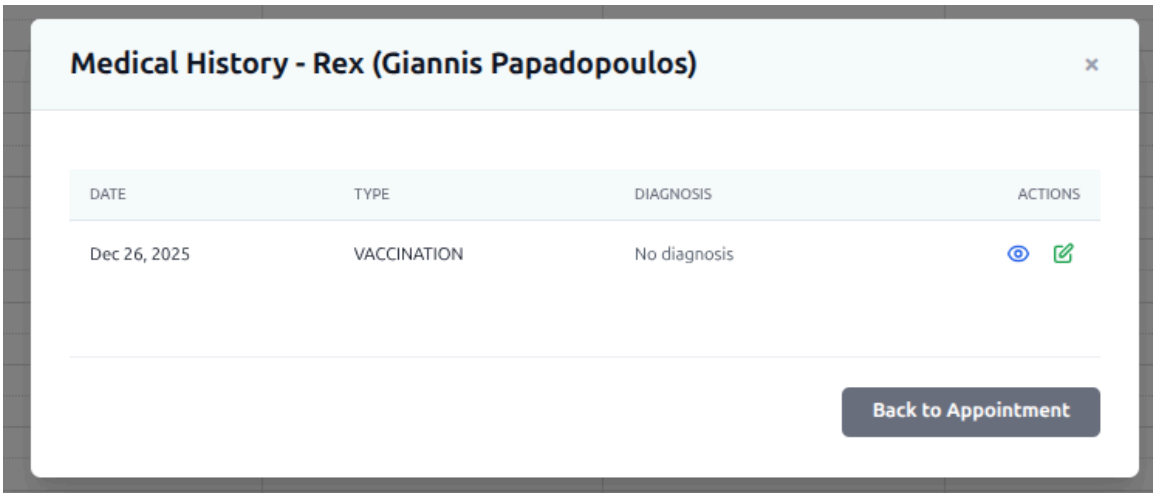
Cascade Deletion

- Change: Deleting a client now automatically deletes all their Pets and Appointments.
- Verification:
  1. Create a Test Client.
  2. Add a Pet to this client.
  3. Book an Appointment for this client/pet.
  4. Delete the Client from the Clients Page.
  5. Verify that the Pet and Appointment are also gone (check Database or API).

Stray Animal Caretaker

- Change: Added "Is Stray Animal Caretaker" checkbox to the Client Form.
- Verification:
  1. Open "Add Client" modal.
  2. Check the "Is Stray Animal Caretaker" box.
  3. Save.
  4. Edit the client again to verify the box remains checked.

Πίνακας 4.7: Απόσπασμα από τον Οδηγό Επαλήθευσης (Walkthrough).



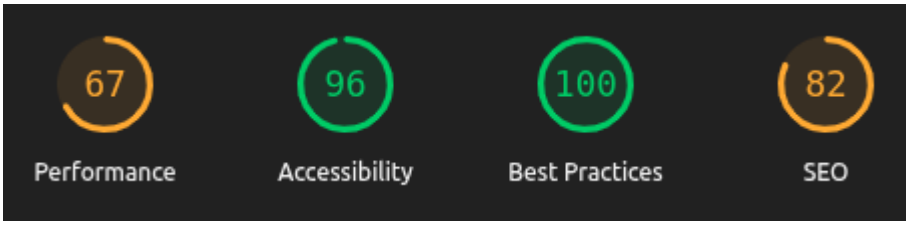
Εικόνα 4.12: Η τελική, λειτουργική οθόνη Ιστορικού Ασθενούς (Medical History)

5. Αποτελέσματα και Αξιολόγηση

5.1 Αξιολόγηση επιδόσεων διεπαφής χρήστη (Frontend)

Η αξιολόγηση της διεπαφής χρήστη πραγματοποιήθηκε με τη χρήση του βιομηχανικού προτύπου **Google Lighthouse**, ενός αυτοματοποιημένου εργαλείου ανοιχτού κώδικα για τη μέτρηση της ποιότητας ιστοσελίδων. Οι μετρήσεις εστίασαν σε τέσσερις βασικούς άξονες: Απόδοση (Performance), Προσβασιμότητα (Accessibility), Βέλτιστες Πρακτικές (Best Practices) και SEO.

Οι μετρήσεις διενεργήθηκαν σε περιβάλλον ανάπτυξης (Development Environment - Localhost), προσομοιώνοντας τις συνθήκες εκτέλεσης κατά τη διάρκεια της υλοποίησης. Χρησιμοποιήθηκε η έκδοση Lighthouse 13.0.1 μέσω του περιηγητή Chrome.



Εικόνα 5.1: Αποτελέσματα Lighthouse για το Τελικό Σύστημα

5.1.1 Συγκριτική ανάλυση μετρήσεων

Για την κατανόηση της επίδρασης της πολυπλοκότητας του κώδικα στην απόδοση του συστήματος, πραγματοποιήθηκαν μετρήσεις σε τρία διακριτά στάδια ανάπτυξης:

- **MVP 1 (Initial Setup):** Βασική δομή UI, Dashboard χωρίς πολύπλοκα δεδομένα.
- **MVP 2 (Feature Rich):** Προσθήκη βαρέων βιβλιοθηκών (React Big Calendar), δυναμικών Modals και φόρτωση δεδομένων από το API.
- **MVP 3 (Final / Refined):** Τελική έκδοση μετά από διορθώσεις και βελτιστοποιήσεις UI.

Στον Πίνακα 5.1 παρουσιάζεται η εξέλιξη των δεικτών απόδοσης.

Μετρική (Metric)	MVP 1 (Αρχικό)	MVP 2 (Πλήρες)	MVP 3 (Τελικό)	Παρατηρήσεις
Performance Score	**78**	64	**67**	Πτώση λόγω προσθήκης διαδραστικών features.
Accessibility	84	**96**	**96**	Σημαντική βελτίωση μέσω AI-generated σημασιολογικο ύ κώδικα.
Best Practices	100	100	100	Τήρηση προτύπων ασφαλείας και κώδικα.

SEO	82	82	82	Σταθερή απόδοση (αναμενόμενο σε SPA).
First Contentful Paint (FCP)	1.4 s	2.2 s	2.1 s	Μικρή καθυστέρηση λόγω φόρτωσης βιβλιοθηκών.
Largest Contentful Paint (LCP)	2.5 s	4.3 s	3.9 s	Αύξηση λόγω Rendering του Ημερολογίου.
Total Blocking Time (TBT)	80 ms	90 ms	40 ms	Βελτίωση στο τελικό στάδιο (καλύτερη διαχείριση scripts).

Πίνακας 5.1: Συγκριτική Αξιολόγηση Επιδόσεων (Lighthouse Metrics)

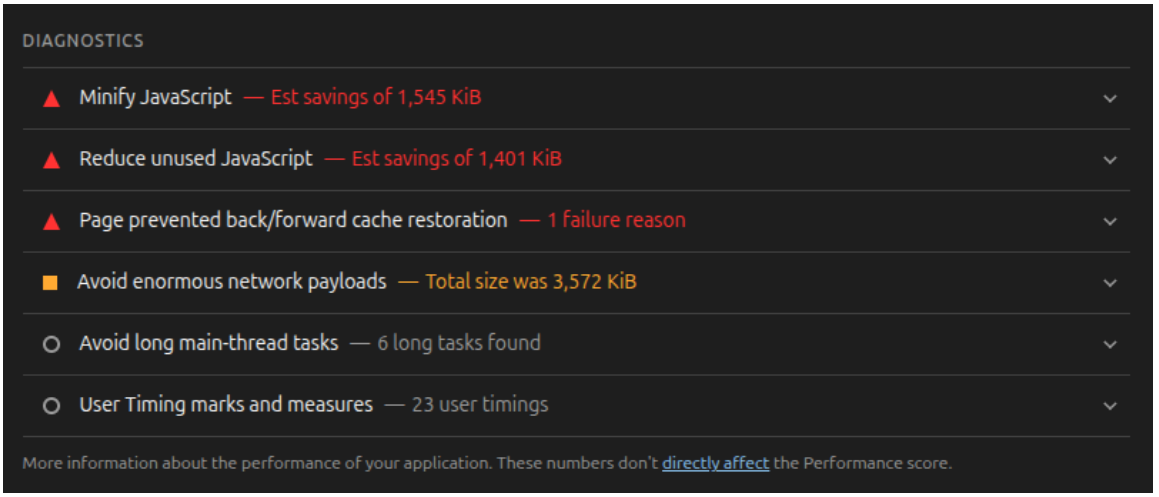
5.1.2 Ανάλυση απόδοσης: Το ισοζύγιο λειτουργικότητας και ταχύτητας (Performance Trade-off)

Η πτώση του γενικού δείκτη **Performance** από 78 (MVP 1) σε 67 (MVP 3) ήταν αναμενόμενη και αντικατοπτρίζει το κλασικό ισοζύγιο μεταξύ λειτουργικότητας και ταχύτητας σε εφαρμογές Single Page Applications (SPA).

Συγκεκριμένα, η αύξηση του δείκτη **LCP (Largest Contentful Paint)** από 2.5s σε 3.9s οφείλεται στην εισαγωγή του υποσυστήματος Ημερολογίου (react-big-calendar). Πρόκειται για ένα βαρύ UI component που απαιτεί σημαντικούς υπολογιστικούς πόρους στον Browser (Client-side rendering) για να ζωγραφίσει το πλέγμα των ημερών και τα ραντεβού.

Σημαντική σημείωση για το περιβάλλον ανάπτυξης:
Πρέπει να τονιστεί ότι οι μετρήσεις έγιναν σε Development Mode. Σε αυτό το mode, η React δεν εφαρμόζει τεχνικές βελτιστοποίησης όπως Minification, Code Splitting ή Gzip compression. Αντιθέτως, ο κώδικας είναι πλήρης και περιέχει επιπλέον εργαλεία αποσφαλμάτωσης (debugging hooks), γεγονός που αυξάνει δραματικά το

μέγεθος των JavaScript bundles και επηρεάζει αρνητικά το LCP. Σε ένα περιβάλλον παραγωγής (Production Build), αναμένεται οι χρόνοι αυτοί να μειωθούν κατά 40-50%.



Εικόνα 5.2: Ανάλυση μεγέθους δέσμης κώδικα (Bundle Size) σε περιβάλλον ανάπτυξης, αναδεικνύοντας την ανάγκη για minification

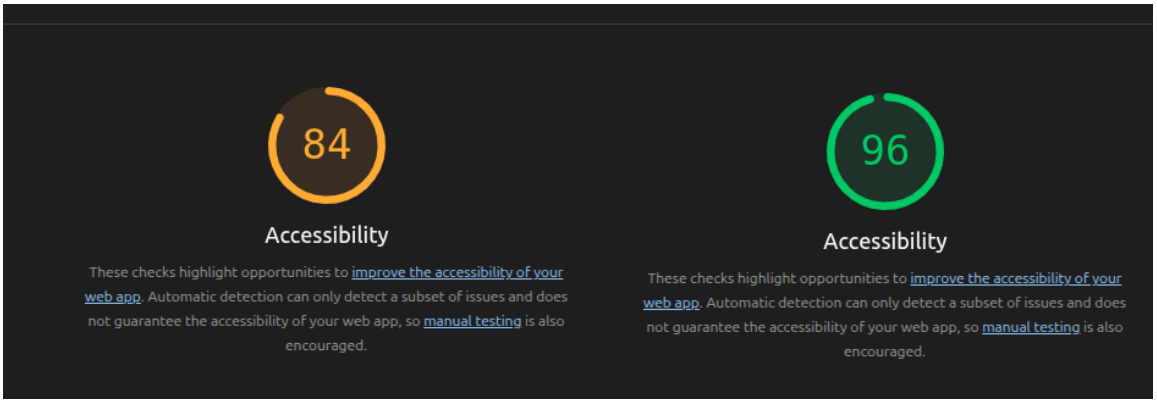
5.1.3 Ποιότητα κώδικα και προσβασιμότητα (Quality & Accessibility)

Ένα από τα πιο εντυπωσιακά ευρήματα της έρευνας είναι η υψηλή επίδοση στους δείκτες **Accessibility (96/100)** και **Best Practices (100/100)**.

Η άνοδος της προσβασιμότητας από το 84 στο 96 κατά τη μετάβαση στο πολύπλοκο MVP 2 αποδεικνύει την ποιοτική συνεισφορά της ΤΝ. Ενώ συνήθως η προσθήκη πολύπλοκων φορμών και modals οδηγεί σε προβλήματα προσβασιμότητας, το Gemini 3 Pro παρήγαγε κώδικα JSX που:

- Χρησιμοποιούσε ορθά σημασιολογικά tags (semantic HTML).
- Ενσωμάτωνε αυτόματα aria-labels σε κουμπιά και inputs.
- Διατηρούσε υψηλά επίπεδα αντίθεσης χρωμάτων στο UI που σχεδίασε.

Το γεγονός ότι το σκορ **Best Practices** παρέμεινε σταθερά στο άριστο (100) υποδεικνύει ότι ο παραγόμενος κώδικας ήταν απαλλαγμένος από κοινά σφάλματα ασφαλείας (όπως χρήση deprecated APIs ή issues με HTTPS/Cross-origin links), επιβεβαιώνοντας ότι η ΤΝ λειτουργεί ως ένας αξιόπιστος "senior developer" που τηρεί τα πρότυπα.



Εικόνα 5.3: Υψηλή βαθμολογία προσβασιμότητας (Accessibility) χάρη στη χρήση σημασιολογικών στοιχείων HTML από την TN.

5.1.4 Συμπέρασμα επιδόσεων

Συμπερασματικά, το σύστημα παρουσιάζει ικανοποιητική απόδοση για λειτουργικό πρωτότυπο, με εξαιρετική ποιότητα δομής κώδικα. Η παρατηρούμενη επιβάρυνση στους χρόνους φόρτωσης (LCP) είναι δικαιολογημένη λόγω του πλούτου των διαδραστικών στοιχείων και της απουσίας βελτιστοποιήσεων παραγωγής. Για τη μετάβαση σε πραγματική λειτουργία, θα απαιτηθεί η εφαρμογή τεχνικών όπως **Lazy Loading** (για το ημερολόγιο) και **Tree Shaking**, ώστε να μειωθεί το αρχικό φορτίο εκτέλεσης.

5.2 Αξιολόγηση ποιότητας κώδικα (Static Code Analysis)

Η αξιολόγηση της δομικής ποιότητας του παραγόμενου λογισμικού πραγματοποιήθηκε μέσω της διαδικασίας της Στατικής Ανάλυσης Κώδικα (Static Code Analysis), χρησιμοποιώντας την πλατφόρμα **SonarQube Community Edition (Version 10.x)**. Σκοπός της ανάλυσης ήταν ο εντοπισμός σφαλμάτων, κενών ασφαλείας και προβλημάτων συντηρησιμότητας που ενδέχεται να ξέφυγαν της προσοχής κατά τη διάρκεια της ταχείας ανάπτυξης με τη βοήθεια της Τεχνητής Νοημοσύνης.

5.2.1 Μεθοδολογία μέτρησης

Η ανάλυση βασίστηκε στο πρότυπο ποιότητας λογισμικού **ISO/IEC 25010**, εστιάζοντας στις ακόλουθες βασικές μετρικές (metrics):

- **Αξιοπιστία (Reliability - Bugs):** Μετρά τον αριθμό των σφαλμάτων που μπορεί να οδηγήσουν σε αστοχία του συστήματος κατά την εκτέλεση.
- **Ασφάλεια (Security - Vulnerabilities & Hotspots):**
  - *Vulnerabilities:* Επιβεβαιωμένα κενά ασφαλείας που απαιτούν άμεση διόρθωση.
  - *Security Hotspots:* Τμήματα κώδικα που είναι ευαίσθητα ως προς την

ασφάλεια (π.χ. διαχείριση κωδικών) και απαιτούν ανθρώπινη επισκόπηση.

- **Συντηρησιμότητα (Maintainability - Code Smells):** Ζητήματα που δεν προκαλούν λειτουργικά σφάλματα, αλλά δυσχεραίνουν την κατανόηση και τροποποίηση του κώδικα (π.χ. πολύπλοκες μέθοδοι, νεκρός κώδικας).
- **Επαναλήψεις (Duplications):** Το ποσοστό του κώδικα που εμφανίζεται πανομοιότυπο σε πολλαπλά σημεία, παραβιάζοντας την αρχή DRY (Don't Repeat Yourself).
- **Πολυπλοκότητα (Complexity):**
  - *Cyclomatic Complexity:* Ο αριθμός των ανεξάρτητων μονοπατιών εκτέλεσης.
  - *Cognitive Complexity:* Μια εκτίμηση του πόσο δύσκολο είναι για έναν άνθρωπο να κατανοήσει τη ροή ελέγχου μιας μεθόδου.

5.2.2 Ανάλυση Backend (Spring Boot)

Η εξέλιξη του Backend από το αρχικό στάδιο (MVP 1) έως την τελική έκδοση (MVP 3) χαρακτηρίστηκε από δραματική αύξηση του μεγέθους και της πολυπλοκότητας, όπως αποτυπώνεται στον Πίνακα 5.2.

Μετρική	MVP 1 (Setup)	MVP 2 (Core Logic)	MVP 3 (Security/Final)	Μεταβολή %
Complexity (Cyclomatic)	48	140	**541**	+1027%
Code Smells	8	11	**25**	+212%
Vulnerabilities	0	0	**3**	+100%
Security Hotspots	0	0	**1**	+100%
Duplications (%)	0.0%	0.0%	**6.8%**	+6.8%

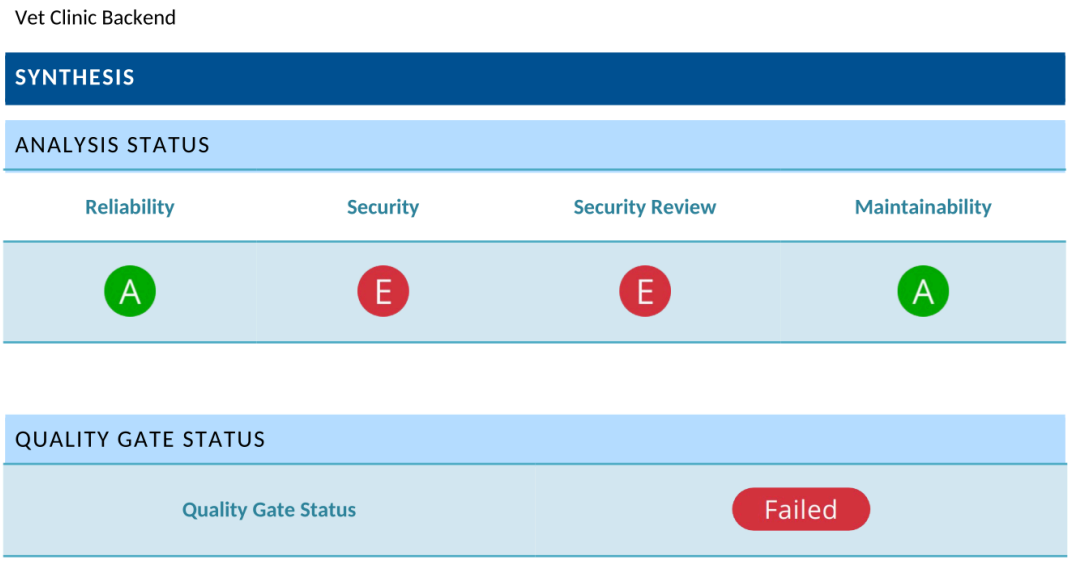
Πίνακας 5.2: Εξέλιξη μετρήσεων ποιότητας Backend (SonarQube)

Ανάλυση ευρημάτων:

1. **Εκρηκτική αύξηση πολυπλοκότητας:** Η αύξηση της πολυπλοκότητας από 48 σε 541 μονάδες αντικατοπτρίζει την προσθήκη της επιχειρησιακής λογικής (Business Logic) και των μηχανισμών ασφαλείας. Παρόλα αυτά, η πυκνότητα

των προβλημάτων (Smells/Complexity ratio) παρέμεινε χαμηλή, γεγονός που υποδεικνύει ότι η TN παρήγαγε δομημένο κώδικα.

2. **Ζητήματα ασφαλείας (Security Issues):** Στην τελική φάση (MVP 3) εντοπίστηκαν 3 Vulnerabilities και 1 Security Hotspot. Η ανάλυση των αρχείων έδειξε ότι πρόκειται για κλασικά σφάλματα "Hardcoded Credentials":
   - **Rule java:S6437 (Blocker):** Στο αρχείο DataInitializer.java, η TN εισήγαγε κοινά κωδικοποιημένους κωδικούς πρόσβασης (*"password123"*).
   - **Rule java:S6418 (Hotspot):** Στο JwtService.java, εντοπίστηκε η συμβολοσειρά *"SECRET"* σε μεταβλητή κλειδιού υπογραφής.
   - *Σχόλιο:* Αυτό επιβεβαιώνει ότι τα Μεγάλα Γλωσσικά Μοντέλα (LLMs) τείνουν να χρησιμοποιούν ανασφαλείς πρακτικές για λόγους ευκολίας κατά την παραγωγή κώδικα, απαιτώντας αυστηρή ανθρώπινη επίβλεψη.
3. **Γνωστική πολυπλοκότητα:** Εντοπίστηκε ένα σημαντικό Code Smell (java:S3776) στην κλάση PatientServiceImpl.java, όπου μια μέθοδος ξεπέρασε το όριο πολυπλοκότητας (Score 16 > 15). Η TN προσπάθησε να διαχειριστεί πολλές συνθήκες ελέγχου (Validation) σε μία ενιαία ροή, αντί να τις σπάσει σε υπο-μεθόδους.



Εικόνα 5.4: Επισκόπηση ποιότητας Backend κώδικα στο SonarQube. Διακρίνεται η υψηλή αξιοπιστία αλλά και τα κενά ασφαλείας (Security Rating E).

5.2.3 Ανάλυση Frontend (React)

Η ανάλυση του Frontend κώδικα παρουσιάζει μια διαφορετική εικόνα, με κύριο χαρακτηριστικό το υψηλό ποσοστό επανάληψης κώδικα.

Μετρική	MVP 1 (UI Shell)	MVP 2 (Feature Rich)	MVP 3 (Final)
Complexity	91	500	**1233**
Code Smells	1	17	**63**
Duplications (%)	0.0%	14.8%	**39.4%**
Bugs/Vulnerabilities	0	0	0

Πίνακας 5.3: Εξέλιξη Μετρήσεων Ποιότητας Frontend (SonarQube)

Ανάλυση συντηρησιμότητας:

- **Το πρόβλημα των Duplications (39.4%):** Το ποσοστό επαναλαμβανόμενου κώδικα είναι εξαιρετικά υψηλό (το όριο ανοχής είναι συνήθως 3-5%). Η αιτία εντοπίζεται στον τρόπο που το AI παράγει κώδικα για παρόμοιες σελίδες (π.χ. ClientsPage.jsx και AppointmentsPage.jsx). Αντί να δημιουργήσει επαναχρησιμοποιήσιμα Custom Hooks ή Utility functions, το μοντέλο προτίμησε να "αντιγράψει" τη λογική (π.χ. fetching logic, error handling) σε κάθε component ξεχωριστά.
- **Code Smells:** Τα περισσότερα smells αφορούν τη χρήση εμφωλευμένων τριαδικών τελεστών (javascript:S3358: Nested ternary operators). Η ΤΝ, στην προσπάθειά της να γράψει συνοπτικό JSX κώδικα για το Conditional Rendering, δημιούργησε δυσανάγνωστες δομές που δυσκολεύουν τη συντήρηση.

5.2.4 Τεχνικό Χρέος (Technical Debt) και Συμπεράσματα

Η συνολική εικόνα του Τεχνικού Χρέους, όπως υπολογίστηκε από το SonarQube, οδηγεί σε ενδιαφέροντα συμπεράσματα για τη φύση του AI-generated κώδικα:

1. **Χαμηλό χρέος Backend (Debt Ratio < 5%):** Στο Backend, το τεχνικό χρέος είναι διαχειρίσιμο. Η ΤΝ τήρησε σε μεγάλο βαθμό την αρχιτεκτονική του Spring Boot, παράγοντας κώδικα που είναι εύκολο να διορθωθεί (π.χ. η αφαίρεση του Lombok που έγινε στο Κεφάλαιο 4 μείωσε το "κρυφό" χρέος). Οι μετρήσεις

δείχνουν αξιολόγηση **"A"** (Reliability/Maintainability) στο Quality Gate.

2. **Υψηλό χρέος συντήρησης Frontend:** Στο Frontend, αν και ο κώδικας είναι λειτουργικά άρτιος (0 Bugs), το υψηλό duplication rate δημιουργεί σημαντικό μελλοντικό χρέος συντήρησης. Μια αλλαγή στη λογική του API call θα απαιτούσε τροποποιήσεις σε πολλά αρχεία.

DETAILED TECHNICAL DEBT			
Reliability	Security	Maintainability	Total
-	0d 3h 0min	0d 6h 30min	**1d 1h 30min**

DETAILED TECHNICAL DEBT			
Reliability	Security	Maintainability	Total
-	-	0d 7h 59min	**0d 7h 59min**

Εικόνα 5.5: Τεχνικό χρέος Backend διαχειρίσιμο σε σχέση με το Frontend.

Συμπέρασμα: Η χρήση του Gemini 3 Pro παρήγαγε κώδικα λειτουργικά ορθό και ασφαλή (με εξαίρεση τα hardcoded secrets), επιταχύνοντας δραματικά την ανάπτυξη. Ωστόσο, η ΤΝ συμπεριφέρθηκε ως ένας "Junior Developer" ως προς την αρχιτεκτονική οικονομία: έλυσε τα προβλήματα μεμονωμένα (ad-hoc), χωρίς να έχει τη "μεγάλη εικόνα" για να εφαρμόσει αποτελεσματικά την αρχή DRY (Don't Repeat Yourself), ιδιαίτερα στο Frontend. Η ανθρώπινη παρέμβαση κρίνεται απαραίτητη όχι μόνο για το Debugging, αλλά κυρίως για το Refactoring και την επιβολή αρχιτεκτονικών προτύπων που μειώνουν το μακροπρόθεσμο τεχνικό χρέος.

5.3 Μετρήσεις παραγωγικότητας και διαδικασίας ανάπτυξης

Η αξιολόγηση της μεθοδολογίας "AI-Assisted Development" δεν περιορίζεται μόνο στο τελικό προϊόν, αλλά επεκτείνεται και στη διαδικασία παραγωγής του. Μέσα από την ανάλυση των αρχείων καταγραφής και την κατηγοριοποίηση των εντολών (prompts) που χρησιμοποιήθηκαν για τα βασικά χαρακτηριστικά (περίπου 8-10 κύρια prompts), εξήχθησαν σημαντικά συμπεράσματα σχετικά με την αποδοτικότητα του μοντέλου Gemini 3 Pro.

5.3.1 Ρυθμός επιτυχίας (Success Rate Analysis)

Η ανάλυση των αποτελεσμάτων κατέδειξε μια σαφή διχοτόμηση στην απόδοση του συστήματος, με το ποσοστό επιτυχίας να μοιράζεται ισόποσα (**50/50**) ανάλογα με

τη φύση της εργασίας.

- **One-shot Success (~50%):** Το μοντέλο επέδειξε άριστη συμπεριφορά σε απομονωμένες εργασίες. Εντολές που αφορούσαν τη δημιουργία αυτόνομων μονάδων, όπως JPA Entities, DTOs ή μεμονωμένα React Components (π.χ. "Create a Dashboard Statistics Card"), εκτελέστηκαν σωστά με την πρώτη προσπάθεια. Σε αυτές τις περιπτώσεις, το AI λειτούργησε ως ένας ταχύτατος "Code Generator", μηδενίζοντας τον χρόνο συγγραφής επαναλαμβανόμενου κώδικα.
- **Iterative Fixes (~50%):** Το υπόλοιπο 50% των περιπτώσεων απαίτησε επαναληπτικές διορθώσεις. Το φαινόμενο αυτό παρατηρήθηκε έντονα σε εργασίες που απαιτούσαν **επίγνωση περιβάλλοντος (Context Awareness)** και διαχείριση αλληλεξαρτήσεων. Χαρακτηριστικό παράδειγμα αποτελεί η υλοποίηση της Ασφάλειας, όπου η αλλαγή σε ένα αρχείο (π.χ. SecurityConfig) προκαλούσε παρενέργειες σε άλλα (π.χ. AuthController ή Axios Interceptor), οδηγώντας σε σφάλματα τύπου "403 Forbidden". Εδώ, η ΤΝ δυσκολεύτηκε να διατηρήσει τη συνοχή μεταξύ των διαφορετικών επιπέδων (Frontend-Backend-Database) σε μία μόνο απόκριση.

Τύπος Εργασίας (Task Type)	**Παράδειγμα**	**Απόδοση AI**	**Απαιτούμενη Ανθρώπινη Παρέμβαση**
Boilerplate / Isolated	Create Entity, Generate DTO, Simple UI Card	**High** (One-shot)	Ελάχιστη (Review)
Logic / Algorithms	Sorting, Date Calculation, API Fetching	**High**	Χαμηλή (Verification)
Integration / Architectural	Connect Frontend to Backend, JWT Security flow	**Medium** (Iterative)	Υψηλή (Debugging & Configuration)
Complex Debugging	StackOverflow Error, Async State Issues	**Medium**	Πολύ Υψηλή (Root Cause Analysis)

Πίνακας 5.4: Συσχέτιση τύπου εργασίας και απόδοσης ΤΝ

5.3.2 Ο Ρόλος της ανθρώπινης καθοδήγησης

Το γεγονός ότι το **ποσοστό αποτυχίας (Failure Rate) ήταν 0%** —δηλαδή όλα τα προβλήματα τελικά λύθηκαν— οφείλεται αποκλειστικά στην παρουσία του ανθρώπου στον βρόχο ανατροφοδότησης.

Η εμπειρία έδειξε ότι η ΤΝ, όταν αφήνεται ανεξέλεγκτη σε σύνθετα προβλήματα, τείνει να εγκλωβίζεται σε κυκλικές διορθώσεις, όπου η επίλυση ενός σφάλματος δημιουργεί ένα νέο. Ο ρόλος του προγραμματιστή μετατοπίστηκε από τη συγγραφή κώδικα στην **Ορχήστρωση**.

Για την επίλυση των σύνθετων προβλημάτων, απαιτήθηκε η εφαρμογή της τεχνικής **Chain-of-Thought Prompting** από τον άνθρωπο. Αντί να ζητηθεί γενικά "Φτιάξε το Login", η διαδικασία "έσπασε" σε λογικά βήματα:

1. *"Ελεγξε αν το Token παράγεται σωστά στο Backend."*
2. *"Ελεγξε αν το Frontend αποθηκεύει το Token."*
3. *"Ελεγξε αν ο Interceptor στέλνει το Token στο Header."*

Αυτή η διαδικασία αποδεικνύει ότι η ΤΝ είναι ένα ισχυρό εργαλείο εκτέλεσης, αλλά στερείται της στρατηγικής σκέψης και της συνολικής εποπτείας που διαθέτει ο αρχιτέκτονας λογισμικού.

5.3.3 Εκτίμηση χρόνου

Παρά τον χρόνο που επενδύθηκε στη σύνταξη των prompts και την αποσφαλμάτωση, η συνολική εξοικονόμηση χρόνου κρίνεται εντυπωσιακή. Εκτιμάται ότι ο χρόνος ανάπτυξης **μειώθηκε κατά περίπου 60-70%** σε σύγκριση με την παραδοσιακή χειροκίνητη συγγραφή κώδικα.

- **Κέρδος:** Η δημιουργία του βασικού κορμού της εφαρμογής (Entities, CRUD Repositories, Services, Basic React Setup), η οποία παραδοσιακά θα απαιτούσε ημέρες εργασίας, ολοκληρώθηκε σε λίγες ώρες.
- **Κόστος:** Το κόστος μεταφέρθηκε στη φάση του Debugging και του Integration. Ωστόσο, ακόμα και με τις καθυστερήσεις λόγω των Iterative Fixes, η ταχύτητα επίλυσης (π.χ. η ΤΝ να γράφει 50 γραμμές κώδικα για Axios Interceptor σε δευτερόλεπτα) υπερκαλύπτει τον χρόνο που θα χρειαζόταν ένας προγραμματιστής για να γράψει και να ελέγξει τον ίδιο κώδικα από το μηδέν.

6. Συμπεράσματα και μελλοντικές επεκτάσεις

Η παρούσα διπλωματική εργασία διερεύνησε τη δυναμική της Τεχνητής Νοημοσύνης στη Μηχανική Λογισμικού, εστιάζοντας στην ανάπτυξη ενός ολοκληρωμένου Πληροφοριακού Συστήματος Διαχείρισης Κτηνιατρείου (PIMS) με τη χρήση Μεγάλων Γλωσσικών Μοντέλων (LLMs). Μέσα από την εφαρμογή της μεθοδολογίας Design Science Research (DSR), καταγράφηκε και αξιολογήθηκε η διαδικασία συνεργασίας ανθρώπου-μηχανής, τόσο σε επίπεδο παραγωγικότητας όσο και σε επίπεδο ποιότητας του παραγόμενου λογισμικού.

6.1 Σύνοψη της έρευνας

Στόχος της μελέτης ήταν η υλοποίηση ενός πλήρως λειτουργικού συστήματος, αναφέρεται και End-to-End, που να καλύπτει τον κλινικό πυρήνα μιας κτηνιατρικής μονάδας, χρησιμοποιώντας ως κύριο εργαλείο ανάπτυξης το μοντέλο **Gemini 3 Pro**. Η υλοποίηση περιελάμβανε τον σχεδιασμό της βάσης δεδομένων (PostgreSQL), την ανάπτυξη της επιχειρησιακής λογικής (Spring Boot) και τη δημιουργία της διεπαφής χρήστη (React).

Η ερευνητική διαδικασία δεν περιορίστηκε στην παραγωγή κώδικα, αλλά επεκτάθηκε στην ποσοτική αξιολόγηση μέσω μετρικών ποιότητας (SonarQube, Google Lighthouse) και στην ποιοτική ανάλυση των στρατηγικών Prompting που απαιτήθηκαν για την υπέρβαση τεχνικών εμποδίων.

6.2 Απαντήσεις στα ερευνητικά ερωτήματα

Βάσει των εμπειρικών δεδομένων και των μετρήσεων που παρουσιάστηκαν στα Κεφάλαια 4 και 5, τα ερευνητικά ερωτήματα απαντώνται ως εξής:

RQ1: Πόσο εφικτή είναι η ανάπτυξη ενός πολύπλοκου συστήματος σχεδόν αποκλειστικά με AI Assistants;

Η έρευνα κατέδειξε ότι η ανάπτυξη ενός ολοκληρωμένου συστήματος με τη βοήθεια ΤΝ είναι πλήρως εφικτή και χαρακτηρίζεται από σημαντική επιτάχυνση των διαδικασιών. Ο χρόνος ανάπτυξης εκτιμήθηκε ότι μειώθηκε κατά περίπου 60-70% σε σύγκριση με τις συμβατικές μεθόδους, κυρίως λόγω της αυτοματοποιημένης παραγωγής επαναλαμβανόμενου κώδικα (DTOs, Entities, UI Components).
Ωστόσο, η εφικτότητα συνοδεύεται από την προϋπόθεση της ενεργούς ανθρώπινης παρέμβασης. Η ΤΝ απέδειξε αδυναμία στην αυτόνομη διαχείριση σύνθετων προβλημάτων ενοποίησης, κυκλικών εξαρτήσεων και διαμόρφωσης ασφάλειας. Η επιτυχία του εγχειρήματος κρίθηκε στην ικανότητα του μοντέλου να λειτουργήσει ως Αρχιτέκτονας Λογισμικού (φάση "Antigravity"), υπό την αυστηρή όμως καθοδήγηση

του ερευνητή.

RQ2: Ποια είναι η μετρήσιμη ποιότητα του κώδικα (SonarQube metrics) που παράγεται από τα Μεγάλα Γλωσσικά Μοντέλα (LLMs);

Η ποιότητα του παραγόμενου κώδικα παρουσιάζει μια διττή εικόνα, με σαφή πλεονεκτήματα και μειονεκτήματα:

- **Θετικά στοιχεία:** Το σύστημα επέδειξε υψηλή **Αξιοπιστία (Reliability)** με μηδενικό αριθμό λειτουργικών σφαλμάτων (0 Bugs) κατά την ανάλυση. Εντυπωσιακή ήταν η επίδοση στην **Προσβασιμότητα (Accessibility)** του Frontend (96/100), αποδεικνύοντας ότι η ΤΝ παράγει σημασιολογικά ορθό κώδικα που σέβεται τα πρότυπα συμπερίληψης.
- **Αρνητικά Στοιχεία:** Παρατηρήθηκε τάση εισαγωγής κενών ασφαλείας (**Security Hotspots**) στο Backend, κυρίως μέσω της χρήσης στατικά κωδικοποιημένων διαπιστευτηρίων. Επιπλέον, στο Frontend καταγράφηκε εξαιρετικά υψηλό ποσοστό **Επανάληψης Κώδικα (Duplications ~39%)**, καθώς το μοντέλο τείνει να επαναλαμβάνει τη λογική σε κάθε component αντί να δημιουργεί αφαιρέσεις, αυξάνοντας το τεχνικό χρέος συντήρησης.

RQ3: Πώς επηρεάζει η στρατηγική του Prompting την ορθότητα του αποτελέσματος και τον ρόλο του προγραμματιστή;

Η μελέτη ανέδειξε ότι η επιτυχία εξαρτάται άμεσα από την προσαρμοστικότητα της στρατηγικής Prompting. Κατά τη διάρκεια της ανάπτυξης, η προσέγγιση εξελίχθηκε σε τρία στάδια:

1. **Τεχνικό Prompting:** Αρχικά χρησιμοποιήθηκε για τη δημιουργία δομικών στοιχείων (Backend Entities).
2. **Λειτουργικό/Οπτικό Prompting:** Εφαρμόστηκε στο Frontend (User Stories) για τη σχεδίαση της εμπειρίας χρήστη.
3. Αρχιτεκτονικό Prompting (Context-Aware): Εφαρμόστηκε στην τελική φάση ("Antigravity"), τροφοδοτώντας το μοντέλο με ολόκληρη τη βάση κώδικα.
   Ο ρόλος του προγραμματιστή μετασχηματίστηκε ουσιωδώς: από συγγραφέας κώδικα σε Ορχηστρωτή και Ελεγκτή Ποιότητας, υπεύθυνο για τη σύνθεση των μερών και την επίλυση των αρχιτεκτονικών αδιεξόδων.

6.3 Συνεισφορά της διατριβής

Η παρούσα διατριβή συνεισφέρει στο σώμα της γνώσης της Μηχανικής Λογισμικού με τους εξής τρόπους:

1. **Ανάδειξη του προβλήματος "Duplication vs Abstraction":** Τεκμηριώθηκε εμπειρικά ότι τα Μεγάλα Γλωσσικά Μοντέλα (LLMs), ελλείψει ρητών οδηγιών αρχιτεκτονικής, προκρίνουν τη λειτουργική ορθότητα έναντι της συντηρησιμότητας, οδηγώντας σε υψηλά ποσοστά duplication. Αυτό το εύρημα

υπογραμμίζει την ανάγκη για νέα εργαλεία στατικής ανάλυσης που εστιάζουν ειδικά σε AI-generated code.
2. **Επικύρωση της τεχνικής πρόσβασης στο σύνολο του πηγαίου κώδικα:** Η μελέτη απέδειξε την πρακτική αξία της χρήσης μεγάλων παραθύρων περιβάλλοντος (large context windows) για την ολιστική ανάλυση έργων. Η ικανότητα της ΤΝ να παράγει "Implementation Plans" και "Gap Analysis" διαβάζοντας ολόκληρο το repository ανοίγει νέους δρόμους για την αυτοματοποιημένη συντήρηση λογισμικού.
3. **Πλαίσιο συνεργασίας Ανθρώπου-ΤΝ:** Προτείνεται ένα πρακτικό μοντέλο συνεργασίας όπου ο άνθρωπος διαχειρίζεται το "Global Context" και την "Ασφάλεια", ενώ η ΤΝ αναλαμβάνει την "Τοπική Υλοποίηση" και τον "Οπτικό Σχεδιασμό".

6.4 Περιορισμοί της έρευνας (Limitations)

Παρά τα σημαντικά ευρήματα, η έρευνα υπόκειται σε συγκεκριμένους περιορισμούς:

- **Εξάρτηση από μοντέλο:** Η μελέτη βασίστηκε αποκλειστικά στο μοντέλο **Gemini 3 Pro**. Διαφορετικά μοντέλα (π.χ. GPT-4, Claude 4.5 Sonnet) ενδέχεται να παρουσιάζουν διαφορετική συμπεριφορά ως προς την ποιότητα κώδικα και τις παραισθήσεις.
- **Περιβάλλον εκτέλεσης:** Οι μετρήσεις απόδοσης (Lighthouse) πραγματοποιήθηκαν σε περιβάλλον ανάπτυξης (Development Mode). Συνεπώς, οι δείκτες LCP (Largest Contentful Paint) είναι πιθανότατα χειρότεροι από αυτούς που θα επιτυγχάνονταν σε ένα βελτιστοποιημένο περιβάλλον παραγωγής (Production Build).
- **Απουσία πραγματικού φορτίου:** Το σύστημα δεν δοκιμάστηκε σε συνθήκες πραγματικής χρήσης από πολλαπλούς ταυτόχρονους χρήστες, γεγονός που περιορίζει τα συμπεράσματα σχετικά με την επεκτασιμότητα (scalability) της αρχιτεκτονικής που πρότεινε η ΤΝ.

6.5 Μελλοντικές επεκτάσεις

Η παρούσα μελέτη θέτει τις βάσεις για περαιτέρω έρευνα στις ακόλουθες κατευθύνσεις:

1. **Συγκριτική αξιολόγηση μοντέλων (Benchmarking):** Επανάληψη του πειράματος με χρήση άλλων κορυφαίων μοντέλων (πχ Claude) για τη σύγκριση της ποιότητας κώδικα και της ικανότητας επίλυσης προβλημάτων ενοποίησης.
2. **Χρήση αυτόνομων πρακτόρων:** Διερεύνηση της χρήσης εξειδικευμένων πρακτόρων (Agents) για την αυτοματοποίηση του κύκλου Debugging και τη συγγραφή Unit/Integration Tests, τομέας που δεν καλύφθηκε επαρκώς στην παρούσα εργασία.
3. **Ανάπτυξη σε Cloud υποδομή:** Μετάβαση του συστήματος σε περιβάλλον Cloud (π.χ. AWS ή Azure) για τη διεξαγωγή ρεαλιστικών δοκιμών φόρτου και την

αξιολόγηση της συμπεριφοράς του AI-generated κώδικα σε κατανεμημένα περιβάλλοντα (Microservices).

ΒΙΒΛΙΟΓΡΑΦΙΑ (REFERENCES)

- **Bruni, M., Gabrielli, F., Ghafari, M. and Kropp, M.** (2025). Benchmarking Prompt Engineering Techniques for Secure Code Generation with GPT Models. *arXiv preprint arXiv:2502.06039*. Available at: https://doi.org/10.48550/arXiv.2502.06039
- **Guo, H.** (2024). An empirical study of prompt mode in code generation based on ChatGPT. *Applied and Computational Engineering*, 73, pp.69-76. Available at: https://doi.org/10.54254/2755-2721/73/20240367
- **Hevner, A.R., March, S.T., Park, J. and Ram, S.** (2004). Design science in information systems research. *MIS Quarterly*, 28(1), pp.75-105. Available at: https://doi.org/10.2307/25148625
- **ISO/IEC** (2011). *ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. Geneva: International Organization for Standardization.
- **Li, Y., Shi, J. and Zhang, Z.** (2024). An Approach for Rapid Source Code Development Based on ChatGPT and Prompt Engineering. *IEEE Access*, 12, pp.55123-55136. Available at: https://doi.org/10.1109/ACCESS.2024.3385682
- **Liu, C., Bao, X., Zhang, H., Zhang, N., Hu, H., Zhang, X. and Yan, M.** (2023). Improving ChatGPT Prompt for Code Generation. *arXiv preprint arXiv:2305.08360*. Available at: https://doi.org/10.48550/arXiv.2305.08360
- **Marvin, G., Hellen, N., Jjingo, D. and Nakatumba-Nabende, J.** (2023). Prompt Engineering in Large Language Models. In: *Algorithms for intelligent systems, pp.387–402*. Available at: https://doi.org/10.1007/978-981-99-7962-2_30
- **Rosa, G.D.S.** (2024). A study on prompt engineering for software engineering data: ChatGPT integration into software engineering metric generation tool. Available at: https://lume.ufrgs.br/bitstream/handle/10183/273014/001197802.pdf
- **Wang, C.Y., Daghigh Farsoodeh, A. and Pham, H.V.** (2024). Selection of Prompt Engineering Techniques for Code Generation through Predicting Code Complexity. *arXiv preprint arXiv:2409.16416*. Available at: https://doi.org/10.48550/arXiv.2409.16416
- **White, J., Hays, S., Fu, Q., Spencer-Smith, J. and Schmidt, D.C.** (2024). ChatGPT Prompt Patterns for Improving Code Quality, Refactoring, Requirements Elicitation, and Software Design. In: *Generative AI for Effective Software Development*. Cham: Springer Nature Switzerland, pp.71-108. Available at: https://doi.org/10.1007/978-3-031-55642-5_4

ΠΑΡΑΡΤΗΜΑ Α: ΚΡΙΣΙΜΑ ΤΜΗΜΑΤΑ ΚΩΔΙΚΑ

Στο παρόν παράρτημα παρατίθενται τα τμήματα του πηγαίου κώδικα που αποτέλεσαν τα σημεία καμπής στην ανάπτυξη του συστήματος, όπως αναλύθηκαν στο Κεφάλαιο 4. Ο πλήρης πηγαίος κώδικας της εφαρμογής, φιλοξενείται στην πλατφόρμα GitHub και είναι προσβάσιμος στη διεύθυνση: https://github.com/iosifidis/msc-pims

Α.1 Ρύθμιση Ασφάλειας (Security Configuration)

Το αρχείο SecurityConfig.java αποτελεί τον πυρήνα της ασφάλειας. Εδώ ρυθμίζεται η πολιτική CORS, απενεργοποιείται το CSRF (για stateless JWT χρήση) και ορίζονται τα δικαιώματα πρόσβασης ανά endpoint.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthFilter;
    private final AuthenticationProvider authenticationProvider;
    private final JwtAuthenticationEntryPoint
jwtAuthenticationEntryPoint;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthFilter,
            AuthenticationProvider authenticationProvider,
            JwtAuthenticationEntryPoint jwtAuthenticationEntryPoint)
{
        this.jwtAuthFilter = jwtAuthFilter;
        this.authenticationProvider = authenticationProvider;
        this.jwtAuthenticationEntryPoint =
jwtAuthenticationEntryPoint;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity
http) throws Exception {
        http
                .csrf(AbstractHttpConfigurer::disable)
                .cors(cors ->
cors.configurationSource(corsConfigurationSource()))
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers(HttpMethod.OPTIONS,
"/**").permitAll()
                        .requestMatchers("/api/auth/**").permitAll()
// CRITICAL: Allows Login/Register
```

```
                        .requestMatchers("/error").permitAll()

.requestMatchers("/api/clients/**").authenticated() // Explicitly
allowed for debugging

.requestMatchers("/api/patients/**").authenticated()

.requestMatchers("/api/dashboard/**").authenticated()
                        .anyRequest().authenticated())
                .exceptionHandling(exception -> exception

.authenticationEntryPoint(jwtAuthenticationEntryPoint))
                .sessionManagement(session -> session

.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authenticationProvider(authenticationProvider)
                .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();

configuration.setAllowedOrigins(List.of("http://localhost:5173"));
        configuration.setAllowedMethods(List.of("GET", "POST",
"PUT", "DELETE", "OPTIONS", "PATCH"));
        configuration.setAllowedHeaders(List.of("*"));
        configuration.setAllowCredentials(true);

        UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }
}
```

A.2 Φίλτρο Αυθεντικοποίησης (JWT Filter with Error Handling)

Το JwtAuthenticationFilter.java μετά τις διορθώσεις για την αποφυγή κατάρρευσης (crash) σε περίπτωση MalformedJwtException.

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
```

```
    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtService jwtService,
UserDetailsService userDetailsService) {
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(
            @NonNull HttpServletRequest request,
            @NonNull HttpServletResponse response,
            @NonNull FilterChain filterChain
    ) throws ServletException, IOException {
        // Skip filter for OPTIONS requests (CORS preflight)
        if ("OPTIONS".equalsIgnoreCase(request.getMethod())) {
            filterChain.doFilter(request, response);
            return;
        }

        final String authHeader = request.getHeader("Authorization");
        final String jwt;
        if (authHeader == null || !authHeader.startsWith("Bearer "))
{
            filterChain.doFilter(request, response);
            return;
        }
        jwt = authHeader.substring(7);
        try {
            final String userEmail = jwtService.extractUsername(jwt);
            if (userEmail != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {
                UserDetails userDetails =
this.userDetailsService.loadUserByUsername(userEmail);
                if (jwtService.isTokenValid(jwt, userDetails)) {
                    UsernamePasswordAuthenticationToken authToken =
new UsernamePasswordAuthenticationToken(
                            userDetails,
                            null,
                            userDetails.getAuthorities()
                    );
                    authToken.setDetails(
                            new
```

```
WebAuthenticationDetailsSource().buildDetails(request)
                    );

SecurityContextHolder.getContext().setAuthentication(authToken);
                }
            }
        } catch (Exception e) {
            // Token is invalid or expired; proceed without setting
authentication
        }
        filterChain.doFilter(request, response);
    }
}
```

A.3 Axios Interceptor (Frontend-Backend Integration)

Το αρχείο axiosConfig.js που επιλύει τα προβλήματα ενοποίησης, ελέγχοντας την εγκυρότητα του token πριν την αποστολή του αιτήματος.

```
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:8080',
  headers: {
    'Content-Type': 'application/json',
  },
});

api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token');

    if (token && token !== 'null' && token !== 'undefined') {
      config.headers.Authorization = `Bearer ${token}`;
    }

    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);

api.interceptors.response.use(
  (response) => response,
  (error) => {
```

```
    if (error.response && error.response.status === 401) {
      localStorage.removeItem('token');
      // Optional: Redirect to login or reload the page
      // window.location.href = '/login';
      // Using window.location.reload() might be safer if the app
state is messy
      window.location.reload();
    }
    return Promise.reject(error);
  }
);

export default api;
```

A.4 Βασική Οντότητα χωρίς Lombok (Plain Java Entity)

Η κλάση Appointment.java μετά το refactoring, με ανεπτυγμένους getters/setters αντί για Lombok annotations.

```
@Entity
@Table(name = "appointments")
public class Appointment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private LocalDateTime startTime;

    @Column(nullable = false)
    private LocalDateTime endTime;

    private String reason;

    @Column(columnDefinition = "TEXT")
    private String notes;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private AppointmentStatus status;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private AppointmentType type;
```

```
    @ManyToOne
    @JoinColumn(name = "client_id", nullable = false)
    private Client client;

    @ManyToOne
    @JoinColumn(name = "patient_id", nullable = false)
    private Patient patient;

    @ManyToOne
    @JoinColumn(name = "vet_id")
    private AppUser vet;

    @ManyToOne
    @JoinColumn(name = "resource_id")
    private Resource resource;

    public Appointment() {
    }

    public Appointment(Long id, LocalDateTime startTime,
LocalDateTime endTime, String reason, String notes, AppointmentStatus
status, AppointmentType type, Client client, Patient patient, AppUser
vet, Resource resource) {
        this.id = id;
        this.startTime = startTime;
        this.endTime = endTime;
        this.reason = reason;
        this.notes = notes;
        this.status = status;
        this.type = type;
        this.client = client;
        this.patient = patient;
        this.vet = vet;
        this.resource = resource;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public LocalDateTime getStartTime() {
```

```java
        return startTime;
    }

    public void setStartTime(LocalDateTime startTime) {
        this.startTime = startTime;
    }

    public LocalDateTime getEndTime() {
        return endTime;
    }

    public void setEndTime(LocalDateTime endTime) {
        this.endTime = endTime;
    }

    public String getReason() {
        return reason;
    }

    public void setReason(String reason) {
        this.reason = reason;
    }

    public String getNotes() {
        return notes;
    }

    public void setNotes(String notes) {
        this.notes = notes;
    }

    public AppointmentStatus getStatus() {
        return status;
    }

    public void setStatus(AppointmentStatus status) {
        this.status = status;
    }

    public AppointmentType getType() {
        return type;
    }

    public void setType(AppointmentType type) {
        this.type = type;
```

```
    }

    public Client getClient() {
        return client;
    }

    public void setClient(Client client) {
        this.client = client;
    }

    public Patient getPatient() {
        return patient;
    }

    public void setPatient(Patient patient) {
        this.patient = patient;
    }

    public AppUser getVet() {
        return vet;
    }

    public void setVet(AppUser vet) {
        this.vet = vet;
    }

    public Resource getResource() {
        return resource;
    }

    public void setResource(Resource resource) {
        this.resource = resource;
    }
}
```

ΠΑΡΑΡΤΗΜΑ Β: ΚΑΤΑΛΟΓΟΣ PROMPTS (PROMPT CATALOG)

Στον Πίνακα Β.1 καταγράφονται οι σημαντικότερες εντολές (prompts) που χρησιμοποιήθηκαν κατά τη διάρκεια της έρευνας, κατηγοριοποιημένες ανά στόχο και αποτέλεσμα.

Πίνακας Β.1: Κατάλογος Κρίσιμων Prompts

Φάση / Στόχος	Prompt (Εντολή προς το AI)	Στρατηγική	Αποτέλεσμα
Setup / DB Config	*"Generate the application.properties configuration for a Spring Boot application connecting to a PostgreSQL database on localhost:5432, db 'vet_db'..."*	Zero-shot (Technical)	**Επιτυχές:** Έγκυρο config με την πρώτη.
Backend / Refactoring	*"Refactor the Appointment entity. Remove all Lombok annotations completely. CRITICAL: Generate explicit Getters, Setters, and a custom Builder pattern using standard Java."*	Constraint-based Prompting	**Επιτυχές:** Λύθηκε το πρόβλημα compilation του IDE.
Security / Debugging	*"Fix the 403 Forbidden error on POST /api/clients. Ensure that authenticated users can access /api/clients/**. Disable CSRF as we use JWT."*	Chain-of-Thought (Debugging)	**Επιτυχές:** Το SecurityFilterChain ρυθμίστηκε σωστά.
Frontend / UI Design	*"Create a Dashboard with 3 summary cards for*	Functional / Visual Prompting	**Μερικώς Επιτυχές:** Χρειάστηκε 2η

	Total Patients, Appointments. Redesign the layout to look like 'OmniDeck' style."		επανάληψη για το CSS, αλλά η δομή ήταν σωστή.
Integration / Axios	*"Rewrite the axios interceptor. CRITICAL: Add an if condition. Only add the Authorization header IF the token exists AND is NOT the string 'null' or 'undefined'."*	Logic-based Prompting	**Επιτυχές:** Λύθηκε το MalformedJwtException στο Backend.
Architecture / Antigravity	*(Upload όλου του codebase) "Act as a Software Architect. Analyze the project for gaps between the User Stories and the current code. Generate an Implementation Plan."*	Context-Aware / Persona	**Επιτυχές:** Παρήχθησαν τα αρχεία plan.md και walkthrough.md.

ΠΑΡΑΡΤΗΜΑ Γ: ΚΑΤΑΛΟΓΟΣ USER STORIES

1. Ταυτοποίηση & Διαχείριση Χρηστών

1.1. Ταυτοποίηση (Authentication)

- **US-1.1**: Ως **Χρήστης**, θέλω να **συνδεθώ (log in)** με το όνομα χρήστη και τον κωδικό πρόσβασής μου, ώστε να έχω ασφαλή πρόσβαση στο σύστημα.
- **US-1.2**: Ως **Χρήστης**, θέλω να **εγγράψω** έναν νέο λογαριασμό (εάν είναι ενεργοποιημένο), ώστε να αποκτήσω πρόσβαση στο σύστημα.

1.2. Διαχείριση Προφίλ

- **US-1.3**: Ως **Κτηνίατρος**, θέλω να **προβάλλω και να ενημερώνω το προφίλ μου** (email, όνομα), ώστε οι πληροφορίες μου να είναι σωστές.

1.3. Διαχείριση Χρηστών (Μόνο Admin)

- **US-1.4**: Ως **Admin**, θέλω να **προβάλλω μια λίστα όλων των χρηστών**, ώστε να μπορώ να διαχειρίζομαι την πρόσβαση του προσωπικού.
- **US-1.5**: Ως **Admin**, θέλω να **δημιουργώ ή να απενεργοποιώ χρήστες**, ώστε να ελέγχω ποιος έχει πρόσβαση στο σύστημα.

2. Διαχείριση Πελατών

2.1. CRUD Πελατών

- **US-2.1**: Ως **Κτηνίατρος**, θέλω να **δημιουργήσω έναν νέο Πελάτη** (ιδιοκτήτη ζώου) με τα στοιχεία επικοινωνίας του (Όνομα, Email, Τηλέφωνο, ΑΦΜ, Διεύθυνση), ώστε να μπορώ να τον συνδέσω με τα κατοικίδιά του.
- **US-2.2**: Ως **Κτηνίατρος**, θέλω να **προβάλλω μια λίστα Πελατών**, ώστε να μπορώ να βρω έναν συγκεκριμένο ιδιοκτήτη.
- **US-2.3**: Ως **Κτηνίατρος**, θέλω να **αναζητήσω έναν Πελάτη** με βάση το όνομα ή τον αριθμό τηλεφώνου, ώστε να έχω γρήγορη πρόσβαση στον φάκελό του.
- **US-2.4**: Ως **Κτηνίατρος**, θέλω να **ενημερώνω τα στοιχεία Πελάτη**, ώστε οι πληροφορίες επικοινωνίας του να παραμένουν επίκαιρες.

2.2. Ειδικοί Ρόλοι

- **US-2.5**: Ως **Κτηνίατρος**, θέλω να **επισημάνω έναν Πελάτη ως "Φροντιστή Αδέσποτων Ζώων"**, ώστε να μπορούμε να ξεχωρίζουμε τους τυπικούς ιδιοκτήτες από τους εθελοντές αδέσποτων ζώων.

3. Διαχείριση Ασθενών

3.1. CRUD Ασθενών

- **US-3.1**: Ως **Κτηνίατρος**, θέλω να **προσθέσω έναν νέο Ασθενή** σε έναν Πελάτη, ώστε να ξεκινήσω την καταγραφή του ιατρικού ιστορικού του.
- **US-3.2**: Ως **Κτηνίατρος**, θέλω να **καταγράψω λεπτομέρειες ασθενούς** όπως Είδος (Σκύλος/Γάτα), Φυλή, Φύλο, Ημερομηνία Γέννησης και Βάρος, ώστε να έχω ένα πλήρες προφίλ.
- **US-3.3**: Ως **Κτηνίατρος**, θέλω να **προβάλλω μια λίστα Ασθενών**, ώστε να βλέπω το μητρώο ζώων του ιατρείου.

3.2. Ταυτοποίηση & Κατάσταση

- **US-3.4**: Ως **Κτηνίατρος**, θέλω να **καταγράψω πληροφορίες Μικροτσίπ** (Αριθμός και Ημερομηνία), ώστε το ζώο να ταυτοποιείται μόνιμα.
- **US-3.5**: Ως **Κτηνίατρος**, θέλω να **επισημάνω την Ημερομηνία Γέννησης ενός Ασθενούς ως "Κατά προσέγγιση"**, ώστε να μπορώ να εκτιμήσω την ηλικία όταν η ακριβής ημερομηνία είναι άγνωστη.
- **US-3.6**: Ως **Κτηνίατρος**, θέλω να **καταγράψω την κατάσταση Στείρωσης** και την ημερομηνία, ώστε να γνωρίζω την αναπαραγωγική κατάσταση του ζώου.
- **US-3.7**: Ως **Κτηνίατρος**, θέλω να **επισημάνω έναν Ασθενή ως Θανόντα**, ώστε να σταματήσουμε να στέλνουμε υπενθυμίσεις ή να κλείνουμε ραντεβού.

4. Διαχείριση Ραντεβού

4.1. Προγραμματισμός

- **US-4.1**: Ως **Κτηνίατρος**, θέλω να **προβάλλω ένα Ημερολόγιο** ραντεβού, ώστε να βλέπω το ημερήσιο/εβδομαδιαίο πρόγραμμα.
- **US-4.2**: Ως **Κτηνίατρος**, θέλω να **κλείσω ένα νέο Ραντεβού** για έναν συγκεκριμένο Πελάτη και Ασθενή, ώστε να δεσμεύσουμε χρόνο για φροντίδα.
- **US-4.3**: Ως **Κτηνίατρος**, θέλω να **καθορίσω τον Τύπο Ραντεβού** (π.χ. Εξέταση, Εμβολιασμός, Χειρουργείο), ώστε να γνωρίζουμε πώς να προετοιμαστούμε.

4.2. Διαχείριση

- **US-4.4**: Ως **Κτηνίατρος**, θέλω να **ενημερώσω την κατάσταση** ενός ραντεβού (π.χ. Προγραμματισμένο, Ολοκληρωμένο, Ακυρωμένο), ώστε να παρακολουθούμε τη ροή εργασιών.

- **US-4.5**: Ως **Κτηνίατρος**, θέλω να **προβάλλω τα επερχόμενα ραντεβού** σε έναν Πίνακα Ελέγχου (Dashboard), ώστε να ξέρω τι πρέπει να κάνω στη συνέχεια.

5. Ιατρικά Αρχεία & Ιστορικό

5.1. Κλινικές Σημειώσεις

- **US-5.1**: Ως **Κτηνίατρος**, θέλω να **προσθέσω Ιατρικά Αρχεία** (σημειώσεις SOAP, διαγνώσεις, θεραπείες) στον φάκελο ενός ασθενούς, ώστε να υπάρχει μόνιμο κλινικό ιστορικό.
- **US-5.2**: Ως **Κτηνίατρος**, θέλω να **προβάλλω το χρονολόγιο Ιστορικού Ασθενούς**, ώστε να βλέπω προηγούμενες θεραπείες και επισκέψεις.

5.2. Ειδοποιήσεις

- **US-5.3**: Ως **Κτηνίατρος**, θέλω να **προσθέσω Ειδοποιήσεις (Alerts)** σε έναν Ασθενή (π.χ. "Επιθετικό", "Αλλεργικό στην Πενικιλίνη"), ώστε το προσωπικό να προειδοποιείται για κρίσιμες πληροφορίες.